



Tunisian Republic
Ministry of Higher Education and Scientific
Research
University of Sousse
Higher Institute of Computer Science
and Communication Technologies
(ISITCOM)



End of Studies Project Report

Submitted in partial fulfillment of the requirements for the
National Engineering Degree — Specialty: **Teleinformatics**

Presented by:

Asma Daoussi

Development of a Digital Asset Marketplace

for Inherited Games Studio (IGS)

Carried out at:



Inherited Games Studio

Academic Supervisor: Mr Sami BEN AMOR

Company Supervisor: Mrs. Nour LTAIEF

Academic Year: 2025–2026

Authorization for Defense

Academic Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mr Sami Beb Amor

Academic Supervisor, ISITCOM — University of Sousse

Signature: _____

Company Supervisor

I hereby authorize the student to submit their End of Studies Project Report for the jury defense.

Mrs Nour Ltaeif

Company Supervisor, Inherited Games Studio

Signature: _____

Dedication

To my beloved family,

*who have been my unwavering pillar of strength
throughout every step of this journey.*

To my parents,

*whose sacrifices, endless encouragement, and unconditional love
have been the foundation upon which I built every dream.*

*To my brother Oussema and my sister Emna,
for the laughter, the support, and the belief
that always pushed me forward.*

To my friends and colleagues,

*who made these long study years memorable
and filled them with moments of joy and solidarity.*

And to myself —

*for the late nights, the persistence, and the courage
to see this through to the end.*

This work is for all of you.

Asma Daoussi

Acknowledgments

First and foremost, I would like to express my sincere gratitude to **Mr. Sami Ben Amor**, my academic supervisor, for his continuous guidance, insightful advice, and unwavering support throughout the realization of this project. His expertise and constructive feedback have been invaluable in shaping both the technical direction and the quality of this work.

I extend my heartfelt thanks to **Mrs Nour Ltaief**, my company supervisor at Inherited Games Studio, for welcoming me into the team, sharing her professional experience, and providing the resources and environment necessary to bring this project to life. Her trust in my capabilities and her constant encouragement made this internship an enriching and rewarding experience.

I would like to express my appreciation to all the staff at **Inherited Games Studio (IGS)** for their warm welcome and their readiness to assist me whenever needed. Working alongside such a talented and passionate team has been a privilege.

My gratitude also goes to the faculty and staff of **ISITCOM — Higher Institute of Computer Science and Communication Technologies** at the **University of Sousse**, whose dedication to quality education provided me with the theoretical and practical foundations that made this project possible.

I am also deeply thankful to the members of the **jury** for dedicating their time to evaluate this work, and for the honor they grant me by participating in its defense.

Finally, I would like to thank my family and friends for their patience, encouragement, and moral support during the long nights and challenging moments of this project.

Asma Daoussi
Sousse, 2026

Contents

Dedication	i
Acknowledgments	ii
List of Figures	vii
List of Tables	viii
List of Acronyms	ix
General Introduction	1
1 General Context	3
1.1 Presentation of the Host Organization	4
1.2 Project Presentation	4
1.2.1 Context and Motivation	4
1.2.2 Problem Statement	5
1.3 Comparative Study of Existing Solutions	5
1.3.1 Analysis	5
1.3.2 Platform Comparison	5
1.3.3 Comparison table of platforms	7
1.3.4 Proposed Solution	7
1.4 Work Methodology	8
1.4.1 Comparative Study of Methodologies	8
1.4.2 Methodology Adopted: Scrum	9
1.4.3 Scrum Process Roles	10
1.4.4 Implementation of the Scrum Methodology	10
1.5 Planning	11
2 Requirements Analysis	12
2.1 Actors Identification	13
2.2 Needs Identification	13
2.2.1 Functional Requirements	13
2.3 Non-Functional Requirements	15
2.4 Project Management with Scrum	15

2.4.1	Product Backlog	15
2.5	Global Use Case Diagrams	17
2.5.1	Marketplace — Use Case Diagram	17
2.5.2	Workspace — Use Case Diagram	18
2.6	Global Class Diagrams	19
2.6.1	Marketplace — Class Diagram	19
2.6.2	Workspace — Class Diagram	20
2.7	Project Architecture: 3-Tier	21
2.8	Other used Design Patterns	21
2.9	Work Environment	22
2.9.1	Hardware Environment	22
2.9.2	Software Environment	22
2.9.3	Frameworks and Libraries	24
2.9.4	Data Persistence and Cloud Services	24
3	Marketplace Foundations	26
3.1	Sprint 1: Authentication, Profiles and RBAC	27
3.1.1	Sprint 1 Backlog	27
3.1.2	Sprint 1 Conception	27
3.1.3	Sprint 1 Realization	31
3.1.4	Sprint 1 Conclusion	33
3.2	Sprint 2: Catalog, Cart, Payment and Library	33
3.2.1	Sprint 2 Backlog	33
3.2.2	Sprint 2 Conception	34
3.2.3	Sprint 2 Realization	38
3.2.4	Sprint 2 Conclusion	40
4	Seller Workflow and Workspace	41
4.1	Sprint 3: Asset Upload, Moderation and Reviews	42
4.1.1	Sprint 3 Backlog	42
4.1.2	Sprint 3 Conception	43
4.1.3	Sprint 3 Realization	47
4.1.4	Sprint 3 Conclusion	49
4.2	Sprint 4: IGS Workspace and Administration	49
4.2.1	Sprint 4 Backlog	49
4.2.2	Sprint 4 Conception	50
4.2.3	Sprint 4 Realization	56
4.2.4	Sprint 4 Conclusion	60

5	Advanced Features	61
5.1	Sprint 5 Backlog	62
5.2	AI Services	62
5.2.1	AI Chatbot	63
5.2.2	Review Sentiment Analysis	66
5.2.3	Personalized Recommendations	68
5.3	Security Hardening	69
5.4	VR/AR Immersive Preview	70
5.4.1	Analysis	70
5.4.2	Realization	71
5.5	Deployment on Hostinger VPS	71
5.5.1	Target Infrastructure	72
5.5.2	Key Deployment Concepts	72
5.5.3	CI/CD Pipeline	72
	General Conclusion	74
	Webography	75
	Abstract	79

List of Figures

1.1	Inherited Games Studio Logo	4
1.2	Unity Asset Store [1]	5
1.3	Fab Marketplace [2]	6
1.4	Sketchfab Platform [3]	6
1.5	CGTrader Marketplace [4]	6
1.6	Conceptual architecture of the proposed solution	8
1.7	Scrum process overview	10
1.8	Scrum Team Roles	10
1.9	ClickUp task board	11
1.10	Project Timeline — 18-week Gantt chart across five Scrum sprints . . .	11
2.1	User Role Hierarchy	13
2.2	Use Case Diagram — IGS Marketplace	18
2.3	Use Case Diagram — IGS Workspace	19
2.4	Class Diagram — IGS Marketplace	20
2.5	Class Diagram — IGS Workspace	20
2.6	Three-Tier Architecture	21
3.1	Use case diagram — Sprint 1	28
3.2	Sequence diagram «Authenticate»	30
3.3	Sequence diagram «Reset Password»	31
3.4	Class diagram — Sprint 1	31
3.5	Interface «Sign in»	32
3.6	Interface «Sign up»	32
3.7	Interface «Profile»	33
3.8	Use case diagram — Sprint 2	35
3.9	Sequence diagram «Purchase an Asset»	37
3.10	Sequence diagram «Download from Library»	38
3.11	Class diagram — Sprint 2	38
3.12	Interface «Catalog»	39
3.13	Interface «Catalog filters»	39
3.14	Interface «Cart»	40
3.15	Interface «Library»	40

4.1	Use case diagram — Sprint 3	43
4.2	Sequence diagram «Upload an Asset»	46
4.3	Sequence diagram «Moderate an Asset»	46
4.4	Class diagram — Sprint 3	47
4.5	Interface «Seller profile»	48
4.6	Interface «Reviews»	48
4.7	Interface «Creator Studio»	49
4.8	Interface «Moderation queue»	49
4.9	Use case diagram — Sprint 4 (Part A)	51
4.10	Use case diagram — Sprint 4 (Part B)	52
4.11	Sequence diagram «Create a Task»	55
4.12	Sequence diagram «Approve Intern Access Request»	55
4.13	Class diagram — Sprint 4	56
4.14	Interface «Workspace dashboard»	56
4.15	Interface «Tasks»	57
4.16	Interface «User management»	57
4.17	Interface «Order management»	58
4.18	Interface «Roles and permissions»	58
4.19	Interface «Audit logs»	59
4.20	Interface «System settings»	59
4.21	Interface «Support inbox»	60
5.1	Sequence diagram «AI Chatbot»	65
5.2	AI chatbot widget — intent-aware assistant available on every page . .	65
5.3	Sentiment analysis — review badges and seller aggregate sentiment summary	67
5.4	Recommendations page — personalized (warm user) and trending (cold start)	69
5.5	WebXR immersive preview — 3D model inspected in VR on Meta Quest 3	71

List of Tables

1.1	Comparison of existing 3D asset marketplace platforms	7
1.2	Comparison of development methodologies	9
2.1	Product Backlog — IGS Marketplace	16
3.1	Sprint 1 Backlog	27
3.2	Use case description: Authenticate	28
3.3	Use case description: Reset Password	29
3.4	Use case description: Assign Role	29
3.5	Sprint 2 Backlog	34
3.6	Use case description: Browse and Filter Catalog	35
3.7	Use case description: Purchase an Asset	36
3.8	Use case description: Download from Library	36
4.1	Sprint 3 Backlog	42
4.2	Use case description: Upload an Asset	44
4.3	Use case description: Moderate an Asset	44
4.4	Use case description: Submit a Review	45
4.5	Sprint 4 Backlog	50
4.6	Use case description: Create a Task	53
4.7	Use case description: Approve Intern Access Request	53
4.8	Use case description: Consult Audit Logs	54
5.1	Sprint 5 Backlog	62
5.2	Intent categories handled by the classifier	64
5.3	Sentiment classification — decision logic	67
5.4	User interaction signals and their weights	68

List of Acronyms

Acronym	Definition
API	Application Programming Interface
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DB	Database
DM	Direct Message
ERP	Enterprise Resource Planning
GLB	GL Binary (3D format)
GLTF	GL Transmission Format
HDRI	High Dynamic Range Imaging
HTTP	HyperText Transfer Protocol
IGS	Inherited Games Studio
ISITCOM	Institut Supérieur de l'Informatique et des Technologies de Communication
JWT	JSON Web Token
MVC	Model-View-Controller
OAuth	Open Authorization
ORM	Object-Relational Mapping
PBR	Physically Based Rendering
PFE	Projet de Fin d'Études (End of Studies Project)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSO	Single Sign-On
UML	Unified Modeling Language
UUID	Universally Unique Identifier

General Introduction

The global video game industry has experienced remarkable growth over the past decade, evolving from a niche entertainment segment into one of the most lucrative creative economies in the world. With revenues surpassing hundreds of billions of dollars annually and a continuously expanding ecosystem of developers, studios, and independent creators, the demand for high-quality digital assets has never been greater. Beyond the games themselves, a significant and often overlooked opportunity lies not in the creation of video games directly, but in building the infrastructure that supports them, namely, digital marketplaces dedicated to the exchange of 3D assets. These platforms serve as essential bridges between creators and consumers, enabling studios, developers, architects, and designers to access a vast library of ready-made models, textures, animations, and environments. The rise of real-time 3D technologies, game engines such as Unity and Unreal Engine, and the expanding metaverse economy has further accelerated the need for structured, accessible, and reliable 3D asset marketplace solutions. Investing in such platforms represents a strategic and high-value proposition in today's digital economy, as they reduce production costs, shorten development cycles, and democratize access to professional-grade visual content.

For this purpose, this project introduces a unified 3D digital marketplace — a centralized platform designed to connect creators and consumers of 3D assets within a single ecosystem. The platform is built as a two-sided system: a public-facing **IGS Marketplace** where external users can browse, upload, purchase, and sell assets across multiple categories, and a private **IGS Workspace** reserved for internal company staff, covering project management, team collaboration, content moderation, and administrative control. By combining both sides under one shared backend, the solution addresses the fragmentation of the 3D asset market while also providing IGS with the internal tooling it needs to operate the platform efficiently.

This report is structured as follows:

- **Chapter 1** presents the host organization, the project context, a comparative study of existing solutions, and the chosen Scrum methodology.
- **Chapter 2** defines the system actors, functional and non-functional requirements, product backlog, and logical architecture.
- **Chapter 3 — Marketplace Foundations** covers Sprints 1 and 2: authentication, RBAC, the public asset catalog, 3D and WebXR previews, cart, payment,

and download library.

- **Chapter 4 — Seller Workflow and Workspace** covers Sprints 3 and 4: asset upload, moderation, reviews, Creator Studio, and the full internal IGS Workspace.
- **Chapter 5 — Advanced Features** covers Sprint 5: the AI chatbot, sentiment analysis, recommendations, security hardening, WebXR immersive preview, and the planned deployment.

General Context

Sommaire

1.1	Presentation of the Host Organization	4
1.2	Project Presentation	4
1.2.1	Context and Motivation	4
1.2.2	Problem Statement	5
1.3	Comparative Study of Existing Solutions	5
1.3.1	Analysis	5
1.3.2	Platform Comparison	5
1.3.3	Comparison table of platforms	7
1.3.4	Proposed Solution	7
1.4	Work Methodology	8
1.4.1	Comparative Study of Methodologies	8
1.4.2	Methodology Adopted: Scrum	9
1.4.3	Scrum Process Roles	10
1.4.4	Implementation of the Scrum Methodology	10
1.5	Planning	11

Introduction

This chapter introduces the host organization and the project context, identifies the problem and reviews existing solutions, then presents the proposed platform and the development methodology adopted.

1.1 Presentation of the Host Organization

Inherited Games Studio (IGS) is a game development studio that creates educational and interactive games using immersive technologies and AI-driven mechanics.

A. Mission

IGS develops games that combine learning with entertainment, using VR, AR and AI to create engaging experiences for users.



Figure 1.1: Inherited Games Studio Logo

B. Methodology

IGS employs several key strategies: immersive VR/AR environments for spatial learning, AI systems that adapt gameplay to player behavior, multiplayer and gamification for engagement, data-driven refinement, and balanced edutainment design.

1.2 Project Presentation

1.2.1 Context and Motivation

The global digital-content economy has entered a phase of sustained growth. Recent market research [35] values the worldwide **3D model marketplace** at approximately **USD 1.92 billion in 2023**, with a forecast of **USD 4.86 billion by 2031** and a compound annual growth rate of **11.2%**. A complementary study [34] attributes most of this demand to three industry verticals: video games (**38.2%** of consumption), architectural visualization (**22.1%**) and film and television production (**17.8%**).

1.2.2 Problem Statement

Despite this global trajectory, Tunisia did not have any dedicated 3D digital asset marketplace. Local developers and studios were forced to rely on foreign platforms.

Beyond the market gap, IGS itself lacked a centralized internal system to manage its own assets, coordinate employee activities, supervise trainees and monitor platform operations. Critical tasks were handled through disconnected tools, creating friction and reducing traceability across the organization.

The project addresses two intertwined problems. The first is external: the absence of a national platform through which Tunisian creators can publish, price and sell three-dimensional content while receiving payment in local currency. The second is internal: the absence within IGS of an integrated management environment that governs asset workflows, access rights, team collaboration and operational visibility under a single, coherent system.

1.3 Comparative Study of Existing Solutions

1.3.1 Analysis

The international landscape is dominated by several major platforms.

1.3.2 Platform Comparison

Unity Asset Store

Launched in 2010, Unity Asset Store is tightly integrated with the Unity Editor, offering millions of assets. However, it is engine-specific, lacks RBAC, and provides no internal team tools.

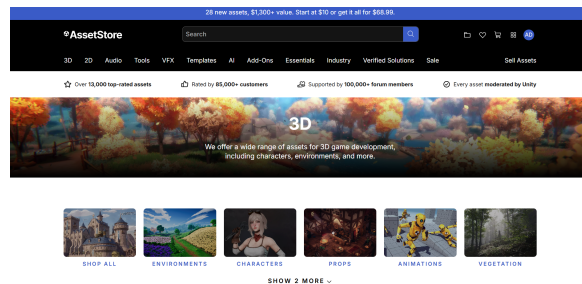


Figure 1.2: Unity Asset Store [1]

Fab (Epic Games)

Fab, Epic Games' unified marketplace, supports multiple engines and benefits from Unreal Engine integration. Despite its breadth, it lacks customization, internal workflows,

and AI-powered features.

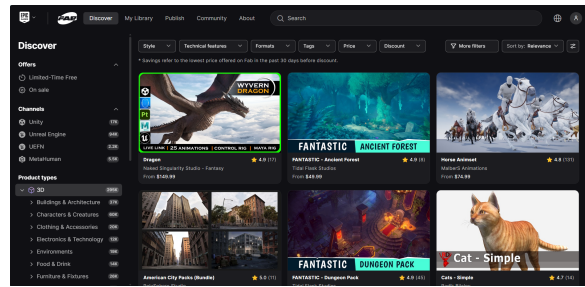


Figure 1.3: Fab Marketplace [2]

Sketchfab

Sketchfab excels in in-browser 3D visualization and supports multiple formats. It is widely used for showcasing models but does not provide monetization flexibility, RBAC, or internal studio management.

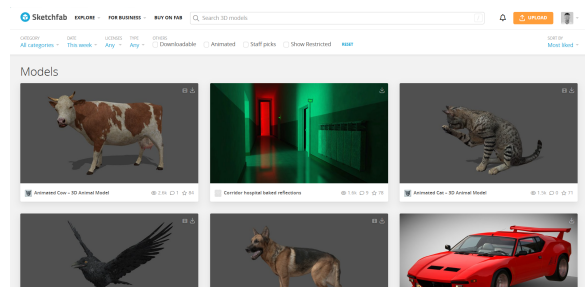


Figure 1.4: Sketchfab Platform [3]

CGTrader

CGTrader offers a large catalog and supports multiple engines. However, it is externally hosted, lacks internal collaboration tools, and does not integrate AI or role-based workflows.

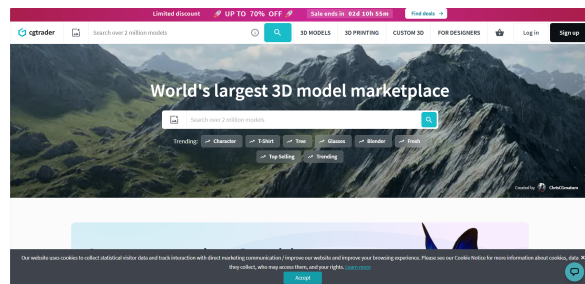


Figure 1.5: CGTrader Marketplace [4]

1.3.3 Comparison table of platforms

Table 1.1: Comparison of existing 3D asset marketplace platforms

Feature	Unity	Fab	Sketchfab	CGTrader	IGS Marketplace
Multi-engine support	×	✓	✓	✓	✓
In-browser 3D preview	Limited	Limited	✓	✓	✓
WebXR / VR-AR preview	×	×	×	×	✓
Custom RBAC	×	×	×	×	✓
Internal team tools	×	×	×	×	✓
AI recommendation	×	×	×	×	✓
AI chatbot	×	×	×	×	✓
Intern access control	×	×	×	×	✓
Local payment gateway	×	×	×	×	✓
Open/self-hosted	×	×	×	×	✓

1.3.4 Proposed Solution

The project developed for IGS consists in a unified digital platform dedicated to the exchange and management of three-dimensional creative assets. It was built as a **two-sided system**.

The first side, called **IGS Marketplace**, is a public-facing storefront open to visitors, buyers and independent creators. It allows external sellers to publish and monetize their work and gives buyers a structured catalog covering thirteen asset categories, with secure transactions and a personal download library. A distinctive feature of this storefront is its **immersive preview capability**: beyond a conventional thumbnail, visitors can manipulate a three-dimensional asset interactively inside the browser and step into the asset in **virtual reality / augmented reality**.

The second side, called **IGS Workspace**, is a private dashboard reserved for company collaborators. It supports the internal life of the studio through project management, team communication, supervised onboarding of trainees, content moderation and access to a full suite of administrative tools. It sits at the intersection of an **ERP** — governing internal processes and resources — and a **CRM** — managing relationships and communication with the platform’s user community.

Although the two interfaces serve different audiences, they share a common foundation that unifies user identity, access-control logic and the data layer. The platform

additionally integrates intelligent components — a conversational assistant, an automated review analyser and a personalized recommendation engine — that enrich the user experience without altering the core business workflows.

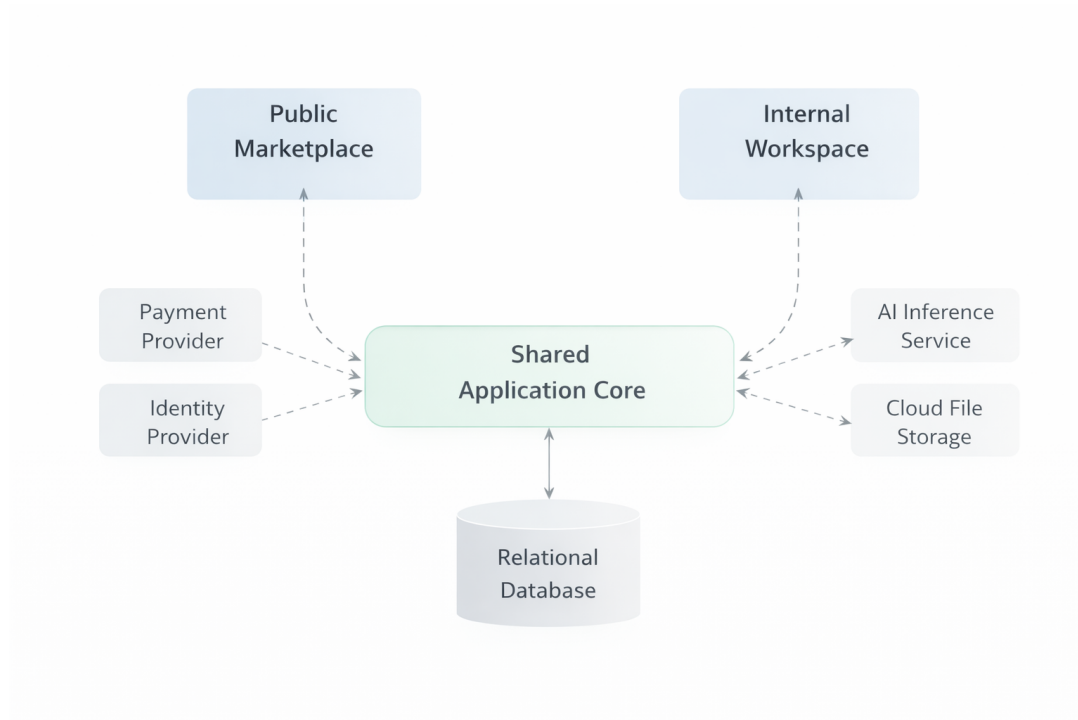


Figure 1.6: Conceptual architecture of the proposed solution

1.4 Work Methodology

1.4.1 Comparative Study of Methodologies

Before selecting a development approach, three widely adopted software engineering methodologies were examined: the Waterfall model, the V-Cycle model, and the Agile Scrum framework. Each carries distinct assumptions about requirements stability, feedback cycles, and risk management.

A. Waterfall Model

The Waterfall model is a **sequential, plan-driven** approach in which each phase (requirements, design, implementation, testing, deployment) must be completed before the next one begins. All requirements are frozen at the outset, making it poorly suited to projects where scope evolves during development.

B. V-Cycle Model

The V-Cycle extends Waterfall by pairing each development phase with a corresponding testing phase. Testing is planned early, improving defect traceability. However, it inherits the same rigidity as Waterfall and delivers a working product only at the end.

C. Agile Scrum Framework

Scrum is an **iterative and incremental** framework built on short development cycles called **sprints** (typically two to four weeks). At the end of each sprint a shippable product increment is delivered and reviewed by stakeholders. Three core roles structure the team: *Product Owner*, *Scrum Master*, and *Development Team*.

D. Side-by-Side Comparison

Table 1.2: Comparison of development methodologies

Criterion	Waterfall	V-Cycle	Agile Scrum
Requirements	Fixed upfront	Fixed upfront	Evolving backlog
Delivery	Single final release	Single final release	Incremental sprints
Stakeholder feedback	At delivery only	At delivery only	Every sprint
Testing integration	End of project	Paired with each phase	Continuous (per sprint)
Flexibility to change	Very low	Low	High
Risk management	Late detection	Earlier via test plans	Continuous mitigation
Best suited for	Stable, well-defined	Safety-critical systems	Evolving, collaborative

1.4.2 Methodology Adopted: Scrum

Scrum was selected for this internship project because it supports adaptive planning, incremental delivery, and regular validation. The application requirements evolved during the internship, and the supervisors were able to review deliverables at the end of each sprint.

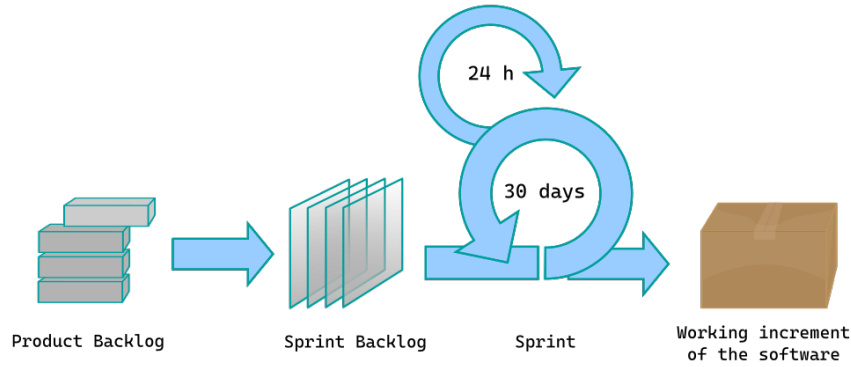


Figure 1.7: Scrum process overview

1.4.3 Scrum Process Roles

The Scrum framework defines clear roles that were adopted for this project:

- **Product Owner:** Owns the product vision, sets priorities, and manages the backlog.
- **Scrum Master:** Facilitates the process, removes impediments, and ensures Scrum practices are followed.
- **Development Team:** Builds the product increment, implements features, and delivers functionality.

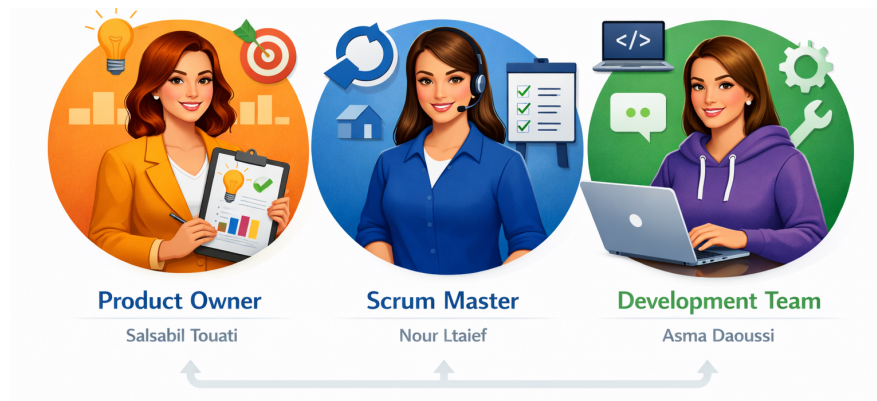


Figure 1.8: Scrum Team Roles

1.4.4 Implementation of the Scrum Methodology

The Scrum methodology was implemented using sprint planning, backlog refinement, and sprint reviews. Tasks were tracked and prioritized using ClickUp.

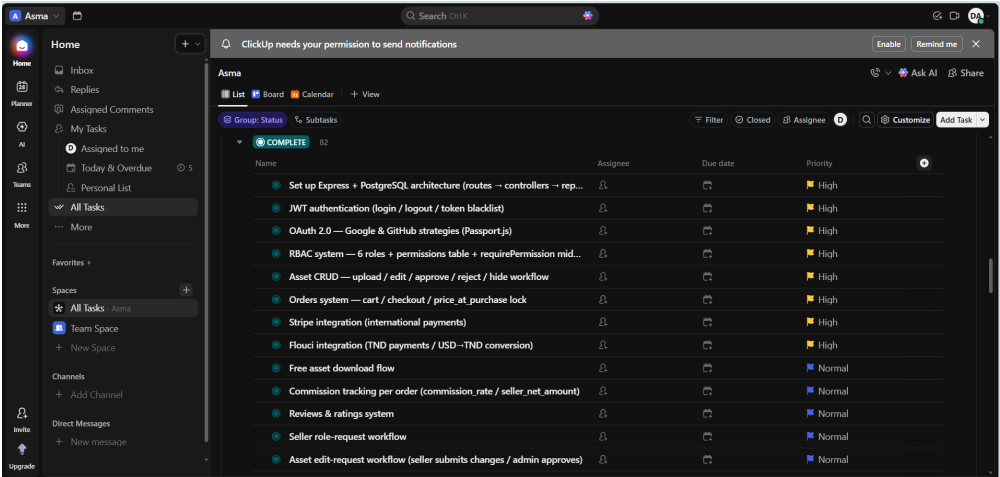


Figure 1.9: ClickUp task board

1.5 Planning

Project planning was driven by sprint sequencing, milestone delivery, and scope tracking. The development timeline spans eighteen weeks across five sprints, with each sprint delivering a coherent slice of functionality.

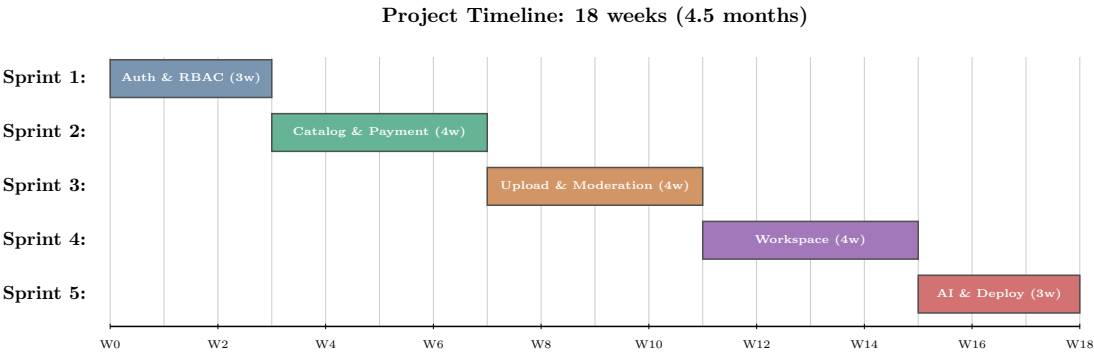


Figure 1.10: Project Timeline — 18-week Gantt chart across five Scrum sprints

Conclusion

This chapter framed the project context, compared existing solutions, and presented the proposed platform and its methodology. The following chapter defines the system requirements and global architecture.

Requirements Analysis

Sommaire

2.1	Actors Identification	13
2.2	Needs Identification	13
2.2.1	Functional Requirements	13
2.3	Non-Functional Requirements	15
2.4	Project Management with Scrum	15
2.4.1	Product Backlog	15
2.5	Global Use Case Diagrams	17
2.5.1	Marketplace — Use Case Diagram	17
2.5.2	Workspace — Use Case Diagram	18
2.6	Global Class Diagrams	19
2.6.1	Marketplace — Class Diagram	19
2.6.2	Workspace — Class Diagram	20
2.7	Project Architecture: 3-Tier	21
2.8	Other used Design Patterns	21
2.9	Work Environment	22
2.9.1	Hardware Environment	22
2.9.2	Software Environment	22
2.9.3	Frameworks and Libraries	24
2.9.4	Data Persistence and Cloud Services	24

Introduction

This chapter identifies the system actors, defines functional and non-functional requirements, and organizes the project scope into a product backlog. It also presents the global use case and class diagrams, the architecture, and the work environment.

2.1 Actors Identification

The platform is used by external marketplace users and by internal IGS collaborators. Access rights are controlled through a role-based access control model (RBAC), where each role receives a set of permissions adapted to its responsibilities.

- **Visitor**
- **Member**
- **Intern**
- **Employee**
- **Administrator**
- **Super Administrator**

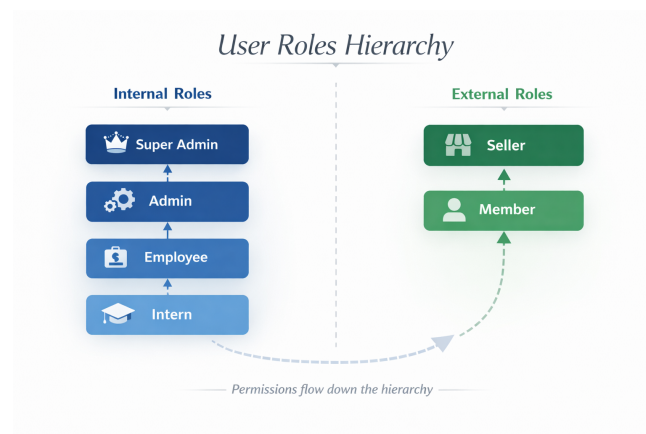


Figure 2.1: User Role Hierarchy

2.2 Needs Identification

2.2.1 Functional Requirements

Common Features :

- **Authentication :** allow users to register, sign in and recover their account.

- **Profile Management** : allow users to view and update their personal information.
- **Asset Browsing** : browse the public catalog by category, search by name, and filter by criteria.
- **Asset Preview** : consult asset details and preview them in the browser or in VR/AR.

Member :

- **Cart & Checkout** : add or remove assets from the cart and proceed to payment.
- **Library** : access and download free or purchased assets.
- **Reviews** : rate, review and like assets.
- **Trade** : propose and respond to peer-to-peer asset exchanges.
- **Seller Request** : submit a request to become a seller.
- **Notifications** : receive notifications about orders, requests and moderation events.

Seller :

- **Asset Upload** : publish assets with files, thumbnails, metadata and pricing.
- **Edit Request** : request modifications on already approved assets.
- **Creator Studio** : consult personal sales statistics and asset performance.
- **Support** : communicate with the moderation team via a ticketing system.

Intern :

- **Asset Request** : request access to paid marketplace assets with a justification.
- **Voucher Redemption** : redeem vouchers issued by authorized staff.
- **Workspace** : collaborate through projects, tasks, and real-time chat.

Employee :

- **Intern Request Management** : approve or reject intern asset access requests.
- **Workspace** : access projects, tasks, performance reports and team chat.

Administrator :

- **User Management** : manage users, roles, permissions and role upgrade requests.
- **Moderation** : approve, reject or hide submitted assets and intern uploads.
- **Platform Management** : manage categories, vouchers, commission rate and settings.
- **Analytics** : consult platform analytics, audit logs and order history.

Super Administrator :

- **Audit Logs** : consult the full audit trail of all platform actions.
- **Role Configuration** : manage roles and their associated permissions.
- **Platform Settings** : control global platform configuration.

2.3 Non-Functional Requirements

Security : the application must guarantee the confidentiality and integrity of user data. Access must be controlled based on roles and permissions, and all sensitive actions must be traceable.

Performance : the application must maintain a minimal response time and remain responsive even with large volumes of content and simultaneous users.

Reliability : the application must always be able to function correctly. Errors must be handled gracefully without disrupting the user experience.

Scalability : the system must be designed to support growth in users, assets and traffic without requiring major structural changes.

Usability : the system must be simple and intuitive to use. Navigation between pages must be smooth, and the interface must be accessible across different devices.

Maintainability : the architecture must make it easy to add new features, fix issues and apply updates without impacting the rest of the system.

2.4 Project Management with Scrum

The project scope is organized as a product backlog of thirty items. Each backlog item represents a user story or technical capability, prioritized by importance and assigned to one of five sprints.

2.4.1 Product Backlog

Table 2.1: Product Backlog — IGS Marketplace

ID	Feature	User Story	Sprint	Priority
1	Authentication	As a user, I can register, sign in, sign out and recover my account securely.	S1	High
2	RBAC	As an administrator, I can assign roles and permissions so each user sees only what they are allowed to.	S1	High
3	User Profile	As a user, I can view and update my personal profile information.	S1	High
4	Catalog	As a visitor, I can browse and filter the public asset catalog by category and keyword.	S2	High
5	Search & Filters	As a visitor, I can narrow results by category, price, rating, file type and availability.	S2	High
6	Asset Preview	As a buyer, I can inspect assets in 3D, VR and AR before purchasing.	S2	High
7	Cart	As a member, I can add, remove and review cart items before checkout.	S2	High
8	Payment & Orders	As a buyer, I can pay securely and receive a confirmed order record.	S2	High
9	Library & Downloads	As a buyer, I can access and download all assets I have unlocked.	S2	High
10	Seller Request	As a member, I can request seller access by providing a justification.	S3	Medium
11	Asset Upload	As a seller, I can upload files, thumbnails, metadata, tags, formats and pricing.	S3	High
12	Moderation	As an admin, I can approve, reject, hide or request changes on submitted assets.	S3	High
13	Asset Edit Request	As a seller, I can request modifications on approved assets without bypassing moderation.	S3	Medium
14	Reviews & Ratings	As a buyer, I can rate and review assets I own.	S3	Medium
15	Notifications	As a user, I can receive notifications about requests, orders and moderation events.	S3	Medium
16	Trades	As a user, I can propose and respond to peer-to-peer asset trades.	S3	Low
17	Intern Requests	As an intern, I can request access to paid assets for production needs.	S4	High
18	Vouchers	As an admin, I can issue vouchers; as a user, I can redeem them.	S4	Medium

ID	Feature	User Story	Sprint	Priority
19	Administration	As an admin, I can manage users, roles, assets, categories, orders and settings.	S4	High
20	Analytics & Audit	As an admin, I can consult analytics and trace actions through audit logs.	S4	Medium
21	Workspace Projects	As internal staff, I can create projects, assign members and monitor progress.	S4	High
22	Workspace Tasks	As internal staff, I can create, assign, update and track tasks.	S4	High
23	Workspace Chat	As internal staff, I can communicate in real time via channels and direct messages.	S4	Medium
24	AI Review Analysis	As an admin, I can use sentiment labels to support moderation decisions.	S5	Medium
25	Recommendations	As a user, I can receive personalized asset recommendations.	S5	Medium
26	Chatbot	As any user, I can ask marketplace questions through the AI assistant.	S5	Medium

2.5 Global Use Case Diagrams

The project is composed of two applications sharing the same backend: the public **IGS Marketplace** and the internal **IGS Workspace**. Each one has a distinct set of actors and capabilities, so a dedicated use case diagram is presented for each.

2.5.1 Marketplace — Use Case Diagram

The figure below presents an overview of the functional behavior of the Marketplace. This diagram represents the interactions between the actors and the use cases of the system. The Marketplace is accessible to all users — both external and internal — who share common capabilities such as browsing the catalog, previewing assets, purchasing and downloading content.

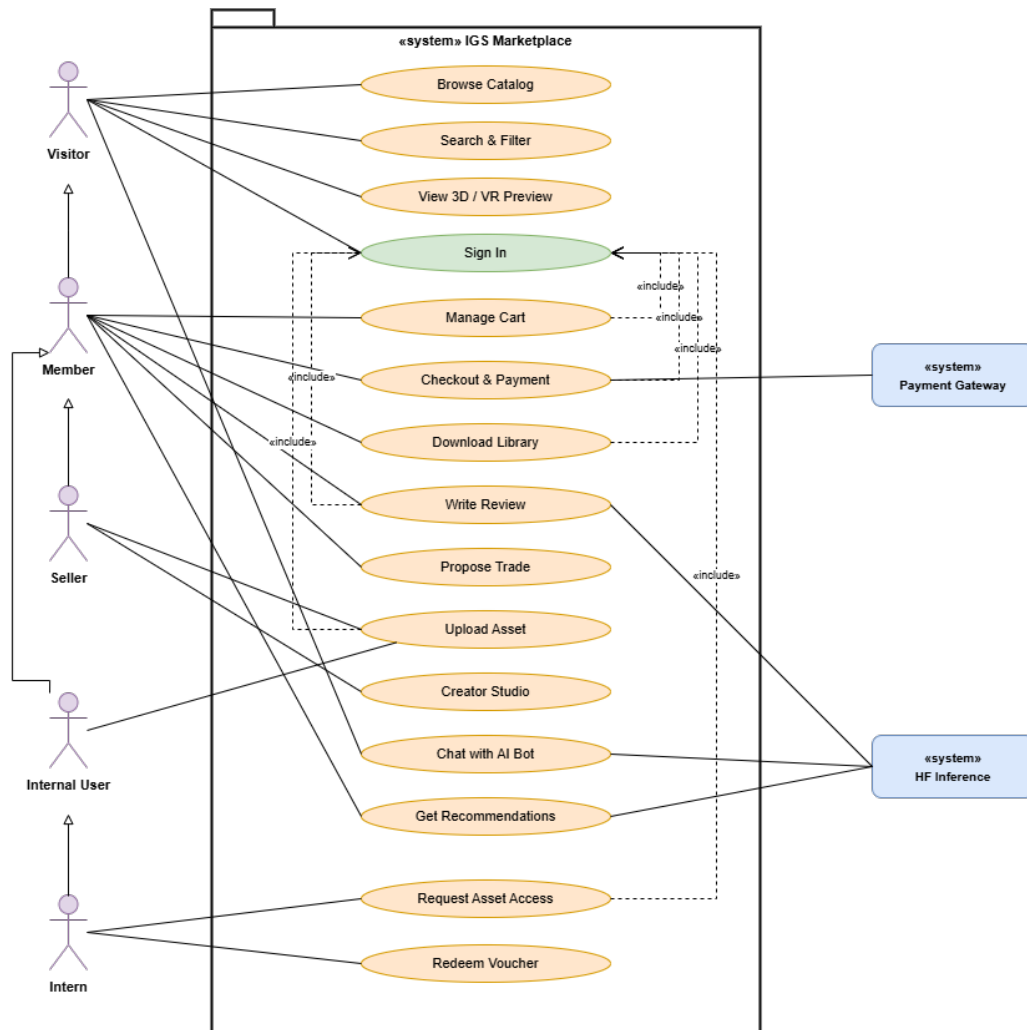


Figure 2.2: Use Case Diagram — IGS Marketplace

2.5.2 Workspace — Use Case Diagram

The Workspace is the internal ERP layer reserved for company staff. The four internal roles are stacked by hierarchy level: Intern (40), Employee (60), Admin (80), Super Admin (100). Each higher role inherits the use cases of the lower one.



Figure 2.3: Use Case Diagram — IGS Workspace

2.6 Global Class Diagrams

Two global class diagrams are presented, one per application. They are derived from the PostgreSQL schema and the backend repositories.

2.6.1 Marketplace — Class Diagram

The Marketplace domain centres on the `User` entity (acting as both buyer and seller), connected to `Asset`, `Order`, `OrderItem`, `CartItem`, `Review`, `Trade` and `InternAssetRequest`.

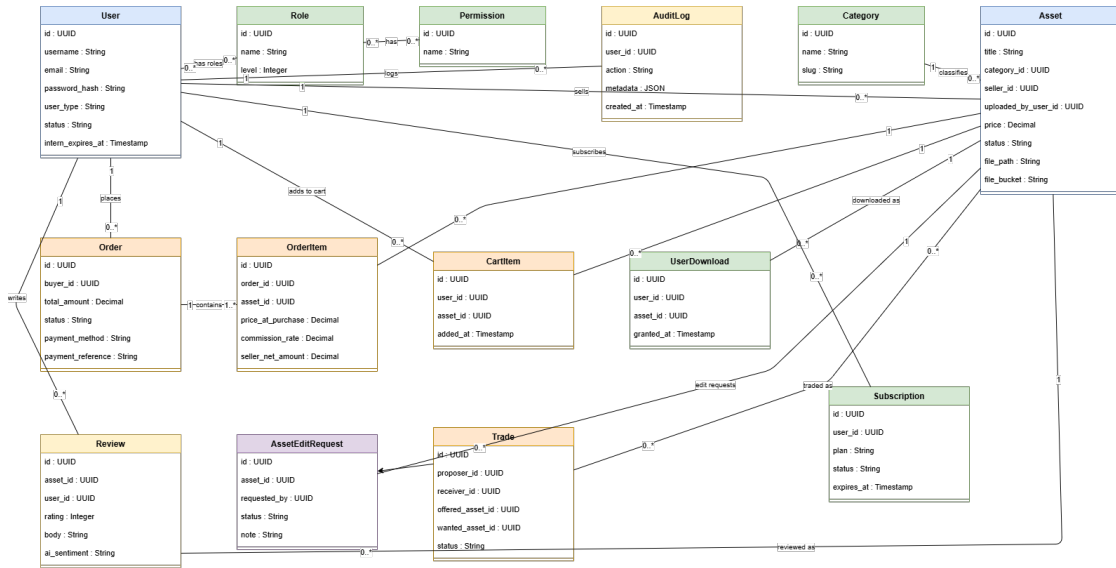


Figure 2.4: Class Diagram — IGS Marketplace

2.6.2 Workspace — Class Diagram

The Workspace domain reuses the same **User** entity but adds the RBAC model (Role, Permission), the project management entities (Project, ProjectMember, WorkspaceTask), the real-time chat (ChatChannel, Message), the Notification feed and the AuditLog trail.

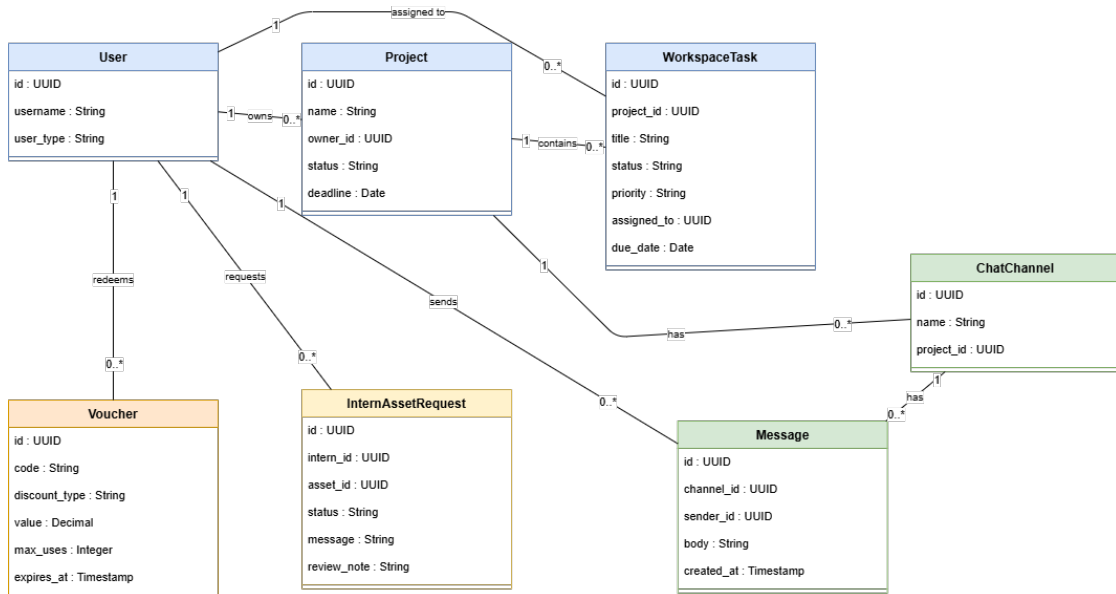


Figure 2.5: Class Diagram — IGS Workspace

2.7 Project Architecture: 3-Tier

The project follows a **three-tier architecture**:

Three-Tier Architecture, also known as three-layer architecture, organizes applications into three logical and physical layers: the presentation layer, the business logic layer, and the data access layer. This separation allows each layer to be developed, maintained, and scaled independently, improving flexibility and reducing the risk of system-wide issues when changes are made.

- **Presentation Layer (UI Layer):** Handles the user interface and interaction, displaying data and capturing user input.
- **Business Logic Layer (Application Layer):** The core of the application where rules, workflows, and data processing occur.
- **Data Access Layer (Persistence Layer):** Manages storage, retrieval, and management of data, abstracting databases and external APIs from upper layers.

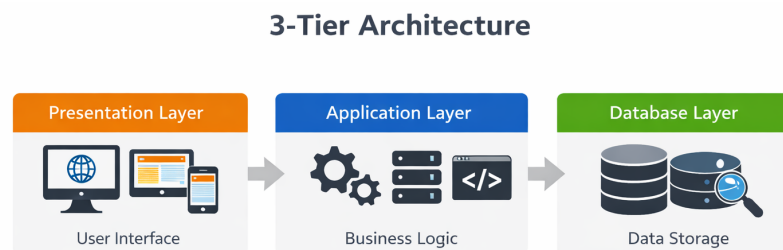


Figure 2.6: Three-Tier Architecture

2.8 Other used Design Patterns

Five core design patterns were applied across the codebase for modularity, testability and maintainability:

Pattern	Purpose
Repository Pattern	Encapsulates all SQL queries in dedicated repository modules; controllers never write SQL directly.
Singleton	Ensures a single shared instance of critical services (database pool, token blacklist, storage client) is reused across the application.
Provider Pattern	Shares state globally across React components without prop drilling; context subscribers receive updates automatically.

2.9 Work Environment

2.9.1 Hardware Environment

The application was developed and tested on a personal laptop with the following specifications:

- **Processor:** Intel Core i7 (8th generation)
- **RAM:** 16 GB DDR4
- **Storage:** 512 GB SSD
- **Operating System:** Windows 11 Pro

For the virtual and augmented reality testing, the following headset was used:

- **Meta Quest 3** [15] A standalone mixed-reality headset manufactured by Meta Platforms, featuring full-colour passthrough cameras for augmented reality and a high-resolution display for virtual reality. It was used throughout the project to validate the WebXR immersive preview feature of the marketplace directly on the target hardware.



2.9.2 Software Environment

The development relied on a carefully selected set of open-source and cloud-based tools spanning the entire delivery chain: code editing, version control, automated integration, database management, design and documentation.

- **Visual Studio Code** [7] A lightweight, open-source source-code editor developed by Microsoft, available on Windows, macOS and Linux. It was the primary development environment for both the frontend and backend codebases, extended with ESLint, Prettier and REST-client plugins.



- **GitHub** [8] A cloud-based platform for Git version control and collaborative software development. The entire project source code was hosted on a private GitHub repository, enabling branch-based development, pull requests and automated CI/CD pipelines.



- **Hostinger** [11] A cloud hosting provider offering Virtual Private Server infrastructure. The backend API and the built frontend assets were deployed on a Hostinger VPS managed by Nginx as a reverse proxy and PM2 as a process supervisor.



- **draw.io** [12] A free, browser-based diagramming tool used to produce the UML diagrams, architecture views and flowcharts included in this report.

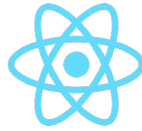


- **ESLint** [13] A static analysis tool for JavaScript that enforces code-style rules and catches common errors before runtime. It was configured with a shared rule-set across both frontends and the backend.



2.9.3 Frameworks and Libraries

- **React** [17] A declarative, component-based JavaScript library for building user interfaces, developed and maintained by Meta. Both the public marketplace and the internal workspace were built as independent React single-page applications.



- **Socket.IO** [25] A bidirectional, event-based communication library built on top of WebSockets. It was used to implement the real-time team chat and the live notification feed shared between the marketplace and the workspace.



- **Express.js** [20] A minimal and flexible Node.js web framework chosen to expose the REST API and compose the security middleware chain covering authentication, authorization, rate limiting and input validation.



2.9.4 Data Persistence and Cloud Services

- **Supabase** [22, 23] An open-source Backend-as-a-Service platform built on PostgreSQL. It was used for cloud file storage: a public bucket for thumbnails and profile pictures, and a private bucket for three-dimensional asset files served through time-limited signed URLs.



- **Stripe** [24] An international payment processing platform used to handle card payments through the Payment Intents API. It was integrated alongside a local Tunisian gateway to support both international and dinar-denominated transactions.



- **Hugging Face** [28] A machine learning platform providing hosted inference APIs. It was used to run the DistilBERT sentiment classification model for automatic review analysis and the Qwen2.5-72B-Instruct large language model that powers the AI chatbot.



Conclusion

This chapter defined the system scope, organized the project into sprints, and established the UML artifacts used throughout the report. The following chapter covers the implementation of the first two sprints.

Marketplace Foundations

Sommaire

3.1 Sprint 1: Authentication, Profiles and RBAC	27
3.1.1 Sprint 1 Backlog	27
3.1.2 Sprint 1 Conception	27
3.1.3 Sprint 1 Realization	31
3.1.4 Sprint 1 Conclusion	33
3.2 Sprint 2: Catalog, Cart, Payment and Library	33
3.2.1 Sprint 2 Backlog	33
3.2.2 Sprint 2 Conception	34
3.2.3 Sprint 2 Realization	38
3.2.4 Sprint 2 Conclusion	40

Introduction

This chapter presents the study and implementation of Sprint 1 and Sprint 2, covering the authentication and access-control foundation, followed by the public marketplace experience.

3.1 Sprint 1: Authentication, Profiles and RBAC

Sprint 1 builds the access foundation. It covers user registration, sign-in, password recovery, profile management and the role-based access control model on which every subsequent feature relies.

3.1.1 Sprint 1 Backlog

Table 3.1 summarizes the backlog of the first sprint.

Table 3.1: Sprint 1 Backlog

ID	User Story	Estimation
1	As a visitor, I want to create an account using email/password or OAuth so I can access protected features.	4 days
2	As a registered user, I want to sign in and receive a JWT so I can call protected endpoints.	3 days
3	As a user, I want to reset my password by email so I can recover my account.	2 days
4	As a user, I want to manage my profile so my public page stays up to date.	2 days
5	As an administrator, I want each user to belong to a role that grants a specific set of permissions.	3 days
6	As an administrator, I want sensitive actions to be logged.	2 days

3.1.2 Sprint 1 Conception

Use Case Diagram

The figure below illustrates the use case diagram relative to Sprint 1.

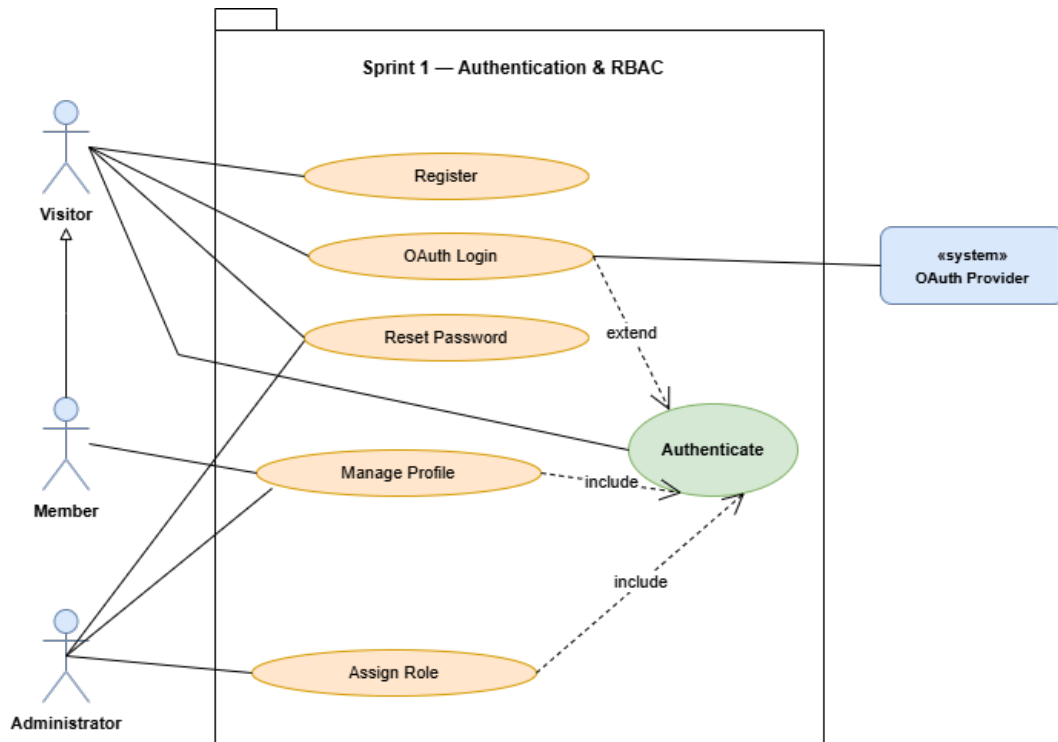


Figure 3.1: Use case diagram — Sprint 1

Table 3.2: Use case description: Authenticate

Use case	Authenticate
Actor	Registered user
Pre-condition	The user has an active account.
Post-condition	The user holds a valid JWT and can access protected routes.
Nominal scenario	1. The user opens the sign-in page. 2. The user enters email and password. 3. The system verifies the credentials. 4. The system signs a JWT and returns it to the browser. 5. The browser stores the token and redirects to the marketplace.
Alternative scenario	3.1. If the credentials are invalid, the system returns an error message. 3.2. Resume at step 2.

Textual description of the use case «Authenticate».

Table 3.3: Use case description: Reset Password

Use case	Reset Password
Actor	Registered user
Pre-condition	The user has lost access to the account but knows the email.
Post-condition	The password is updated and the user can sign in again.
Nominal scenario	1. The user requests a reset link by entering an email. 2. The system generates a reset token and emails the reset link. 3. The user opens the link and submits a new password. 4. The system validates the token and stores the new hashed password. 5. A confirmation message is displayed.
Alternative scenario	4.1. If the token is expired or invalid, an error message is shown. 4.2. The user requests a new reset link.

Textual description of the use case «Reset Password».

Table 3.4: Use case description: Assign Role

Use case	Assign Role
Actor	Administrator
Pre-condition	The administrator is authenticated.
Post-condition	The target user has the chosen role and the associated permissions.
Nominal scenario	1. The administrator opens the user management page. 2. The administrator selects a user. 3. The administrator chooses a role from the list. 4. The system updates the role assignment and logs the action. 5. A success message is displayed.
Alternative scenario	3.1. If the administrator does not have the permission to assign the chosen role, an error is shown. 3.2. The administrator picks a different role.

Textual description of the use case «Assign Role».

Sequence Diagrams

Sequence diagram «Authenticate». When a user attempts to authenticate, the browser sends the credentials to the `AuthController`. The system looks up the user record, verifies the password hash and signs a JWT. The token is returned to the browser, which stores it and attaches it as a Bearer header on every subsequent call. If credentials are invalid, the system returns an error and no token is issued.

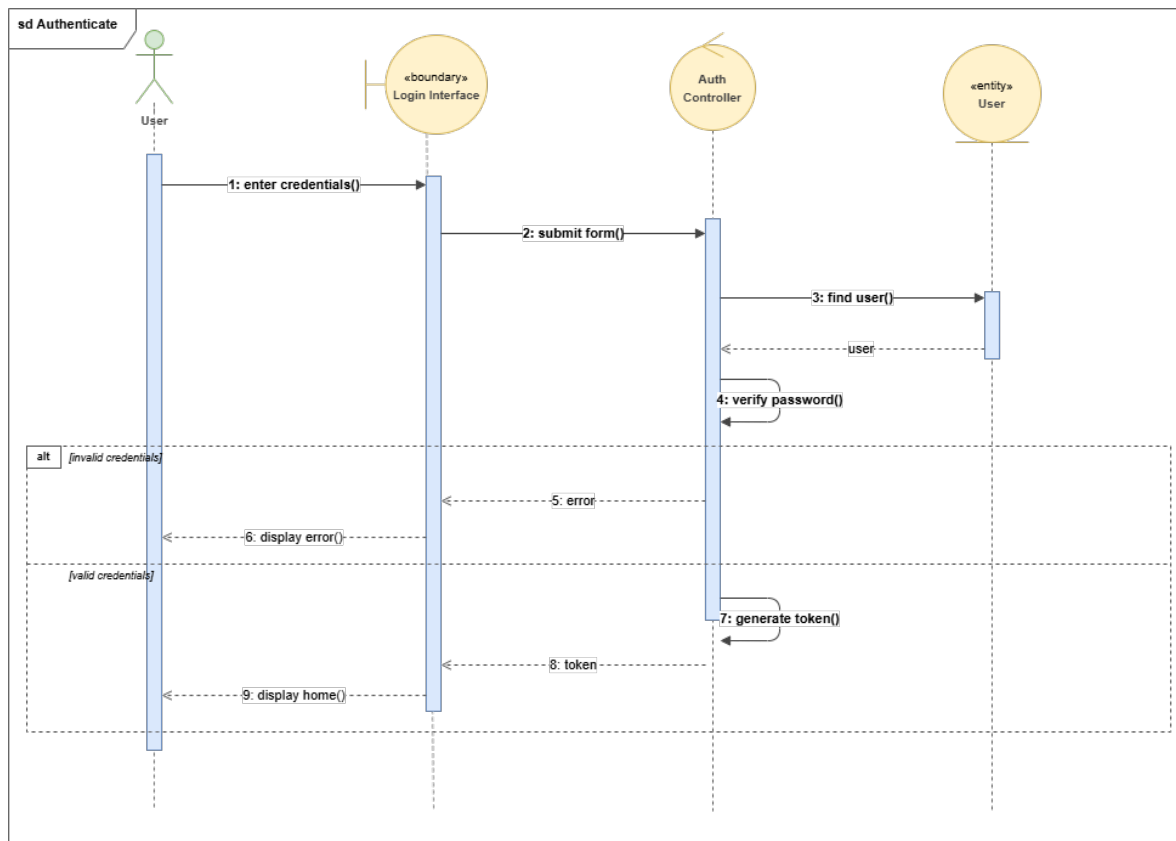


Figure 3.2: Sequence diagram «Authenticate»

Sequence diagram «Reset Password». When a user requests a password reset, the system generates a unique token, stores it on the user record with an expiry, and sends a reset link by email. When the user opens the link and submits a new password, the system verifies the token, hashes the new password and clears the reset token. If the token is missing or expired, the system rejects the request.

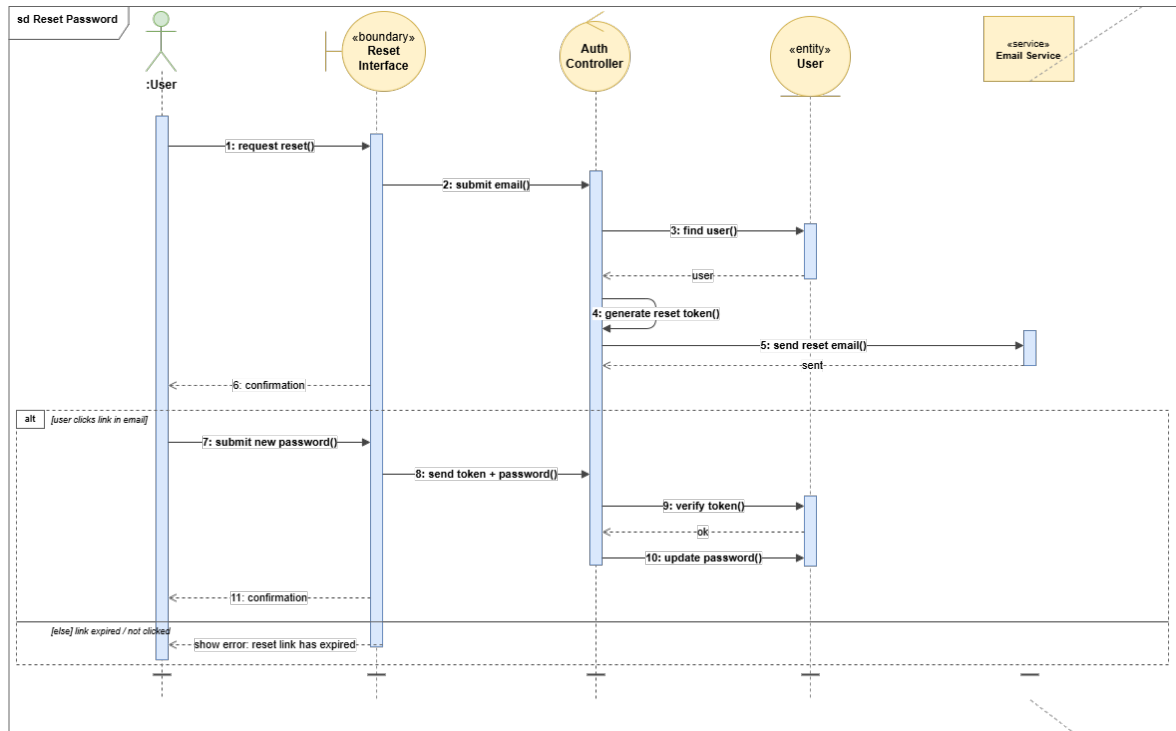


Figure 3.3: Sequence diagram «Reset Password»

Class Diagram

The class diagram of Sprint 1 centres on the **User** entity linked to the RBAC trio **Role**, **Permission** and **AuditLog**. A user holds one or more roles, each role carries a set of permissions, and every sensitive action is traced in the audit log.

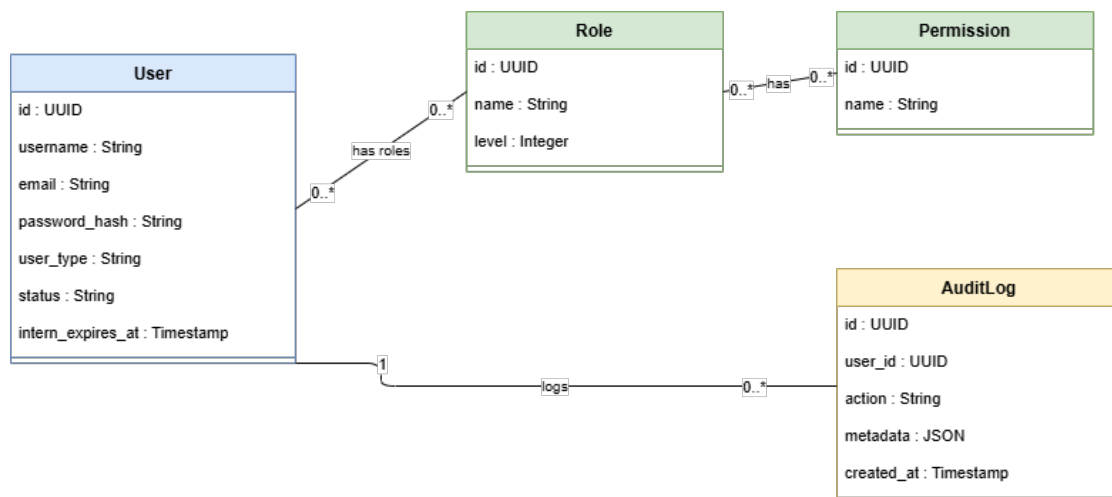


Figure 3.4: Class diagram — Sprint 1

3.1.3 Sprint 1 Realization

In this section we present the interfaces developed during Sprint 1.

«**Sign-in interface**». This interface allows the user to authenticate securely through email and password or through an OAuth provider. On success, the system stores the JWT and redirects to the marketplace.

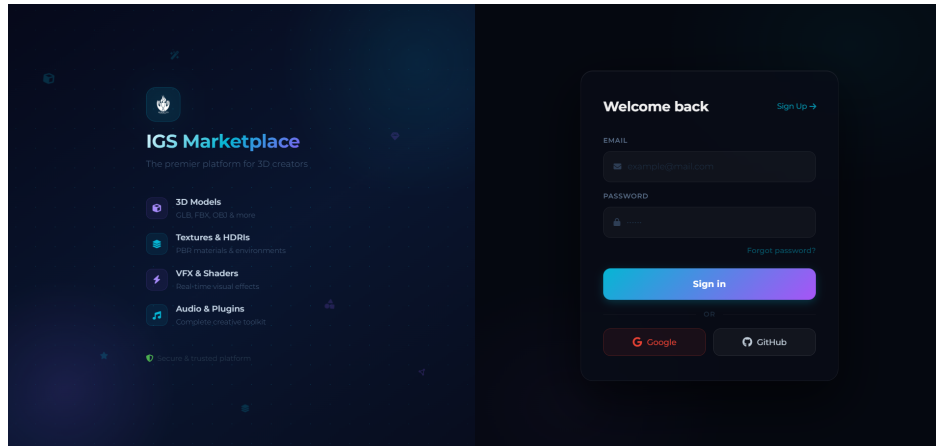


Figure 3.5: Interface «Sign in»

«**Sign-up interface**». This interface lets a new visitor create an account by filling in a short form. Validation is enforced on the email format and the password strength.

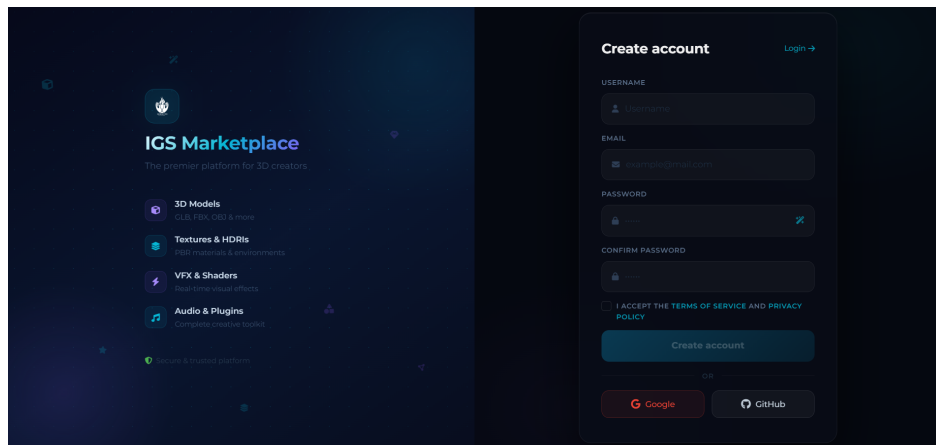


Figure 3.6: Interface «Sign up»

«**Profile management**». This interface allows the authenticated user to view and update personal information (avatar, display name, bio, contact details) and to change the account password.

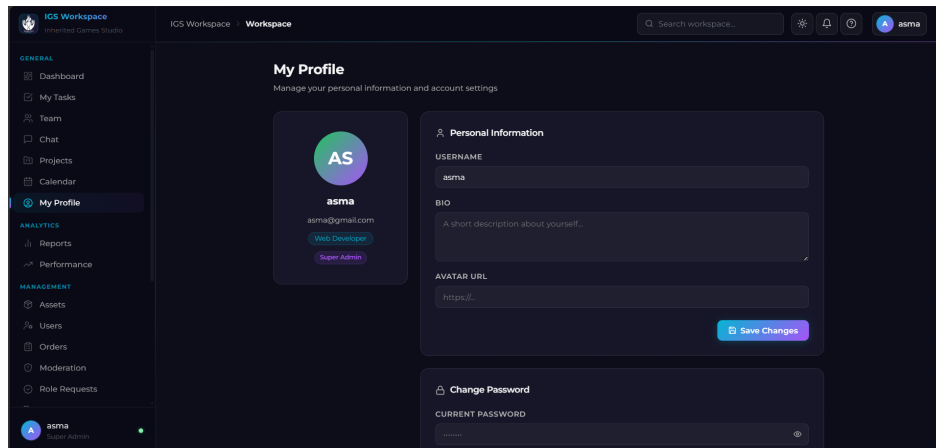


Figure 3.7: Interface «Profile»

3.1.4 Sprint 1 Conclusion

At the end of Sprint 1 the platform has a working authentication layer with JWT and OAuth, a profile management page, and a hierarchical RBAC model with six roles. This foundation conditions every feature delivered in the following sprints.

3.2 Sprint 2: Catalog, Cart, Payment and Library

Sprint 2 turns the platform into a usable storefront. It delivers the filterable asset catalog, the interactive 3D and WebXR previews, the shopping cart, the Stripe and Flouci payment flows and the personal download library.

3.2.1 Sprint 2 Backlog

Table 3.5 summarizes the backlog of the second sprint.

Table 3.5: Sprint 2 Backlog

ID	User Story	Estimation
1	As a visitor, I want to browse a paginated catalogue with filters by category, price and tags.	3 days
2	As a visitor, I want to consult an asset detail page with a 3D preview adapted to the asset type.	4 days
3	As a member, I want to enter VR / immersive mode for compatible 3D assets so I can inspect them at scale.	2 days
4	As a member, I want to add or remove assets from a personal cart.	2 days
5	As a member, I want to pay for the cart through Stripe or Flouci and receive instant access.	4 days
6	As a member, I want a personal library listing all the assets I can download, with signed-URL downloads.	3 days

3.2.2 Sprint 2 Conception

Use Case Diagram

The figure below illustrates the use case diagram relative to Sprint 2.

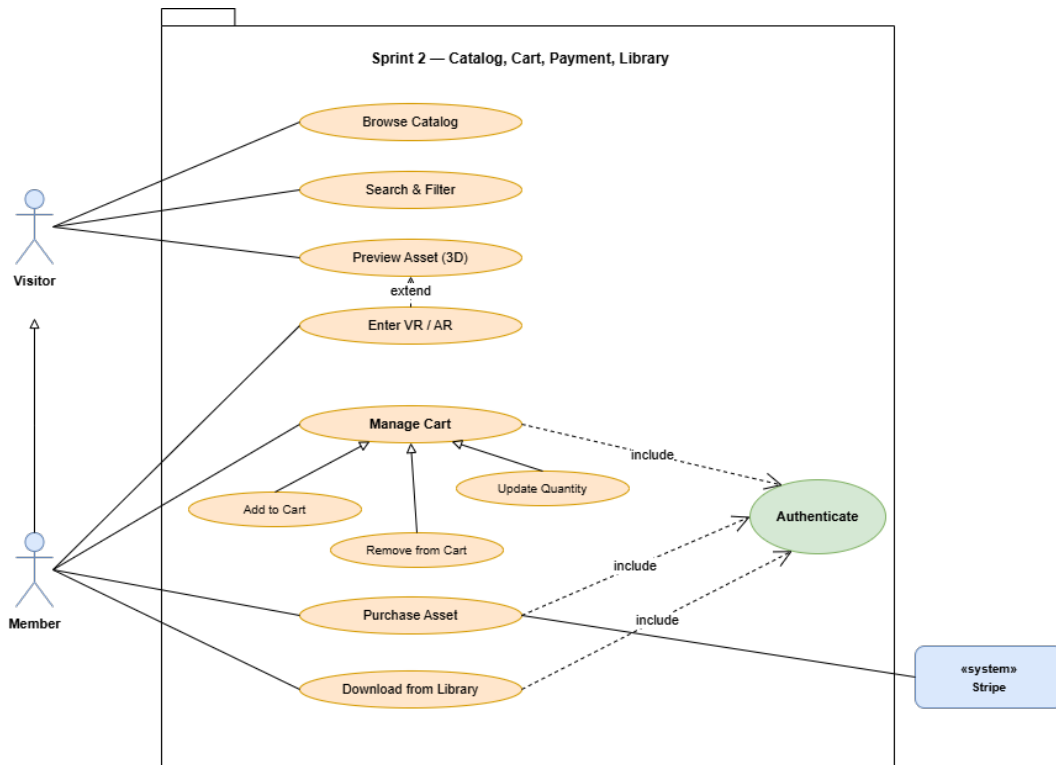


Figure 3.8: Use case diagram — Sprint 2

Table 3.6: Use case description: Browse and Filter Catalog

Use case	Browse and Filter Catalog
Actor	Visitor / Member
Pre-condition	The catalog page is accessible.
Post-condition	The catalog displays the filtered list of approved assets.
Nominal scenario	1. The visitor opens a category page. 2. The visitor applies filters on price, rating or tags. 3. The system queries the catalog with the selected filters. 4. The system returns a paginated list of matching assets. 5. The visitor clicks an asset card to open the detail page.
Alternative scenario	4.1. If no asset matches the filters, an empty-state message is displayed.

Textual description of the use case «Browse and Filter Catalog».

Table 3.7: Use case description: Purchase an Asset

Use case	Purchase an Asset
Actor	Member
Pre-condition	The member is authenticated and has at least one asset in the cart.
Post-condition	The order is recorded as paid and the asset is available in the library.
Nominal scenario	1. The member opens the cart and clicks Checkout. 2. The member selects Stripe or Flouci as payment method. 3. The system creates a payment intent and presents the card form. 4. The payment gateway confirms the transaction. 5. The system records the order, copies the cart items and unlocks the library. 6. A confirmation message is shown.
Alternative scenario	4.1. If the payment fails, the order stays pending and the library is not unlocked. 4.2. The member can retry from the cart.

Textual description of the use case «Purchase an Asset».

Table 3.8: Use case description: Download from Library

Use case	Download from Library
Actor	Member
Pre-condition	The member has at least one asset unlocked in the library.
Post-condition	The asset file is downloaded to the user's machine.
Nominal scenario	1. The member opens the library page. 2. The member clicks Download on an asset. 3. The system re-checks the access level for the asset. 4. The system generates a signed URL with a one-hour validity. 5. The browser fetches the file directly from cloud storage.
Alternative scenario	3.1. If the access level is blocked, the system returns a 403 error.

Textual description of the use case «Download from Library».

Sequence Diagrams

Sequence diagram «Purchase an Asset». When a member proceeds to checkout, the backend creates a payment intent at the gateway and returns the client secret to the browser. The user confirms the card and the gateway notifies the backend of success. The backend then opens a transaction, inserts the order, copies the cart items into `order_items` with the locked commission split, grants library access, and returns the confirmation to the browser.

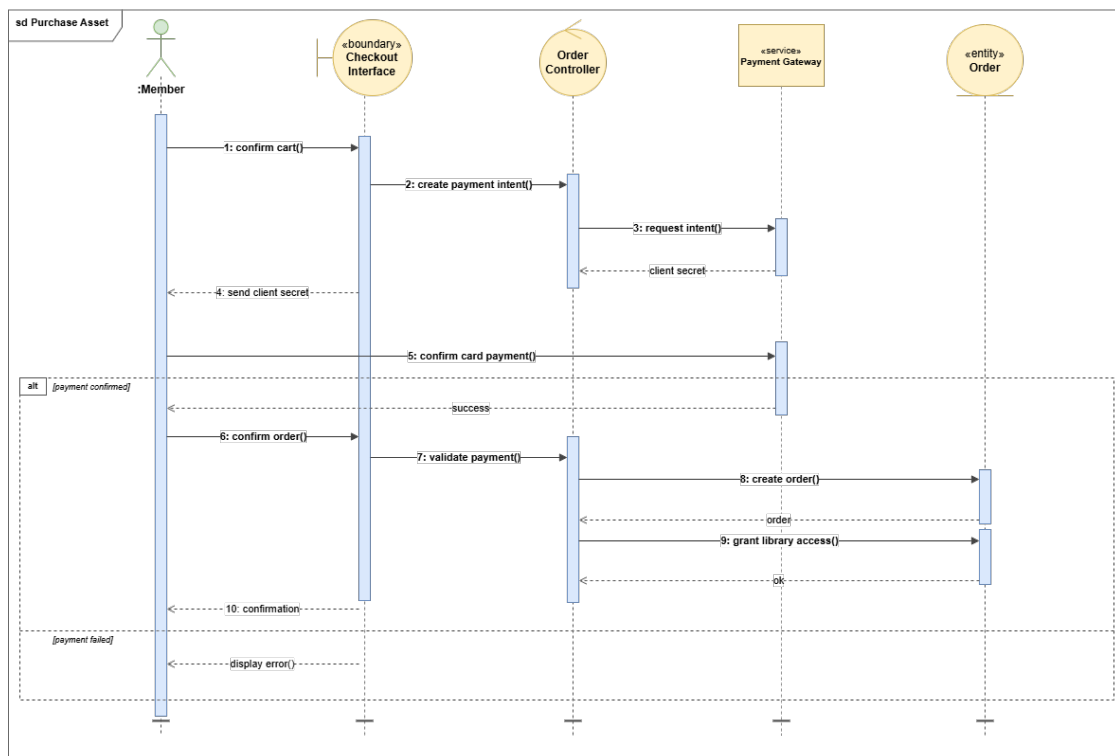


Figure 3.9: Sequence diagram «Purchase an Asset»

Sequence diagram «Download from Library». When a member clicks Download, the backend re-checks the access level for the asset. If the access is granted, the backend retrieves the file path and the bucket and generates a one-hour signed URL through the storage service. The browser uses this signed URL to fetch the file directly from the CDN, without exposing the bucket path.

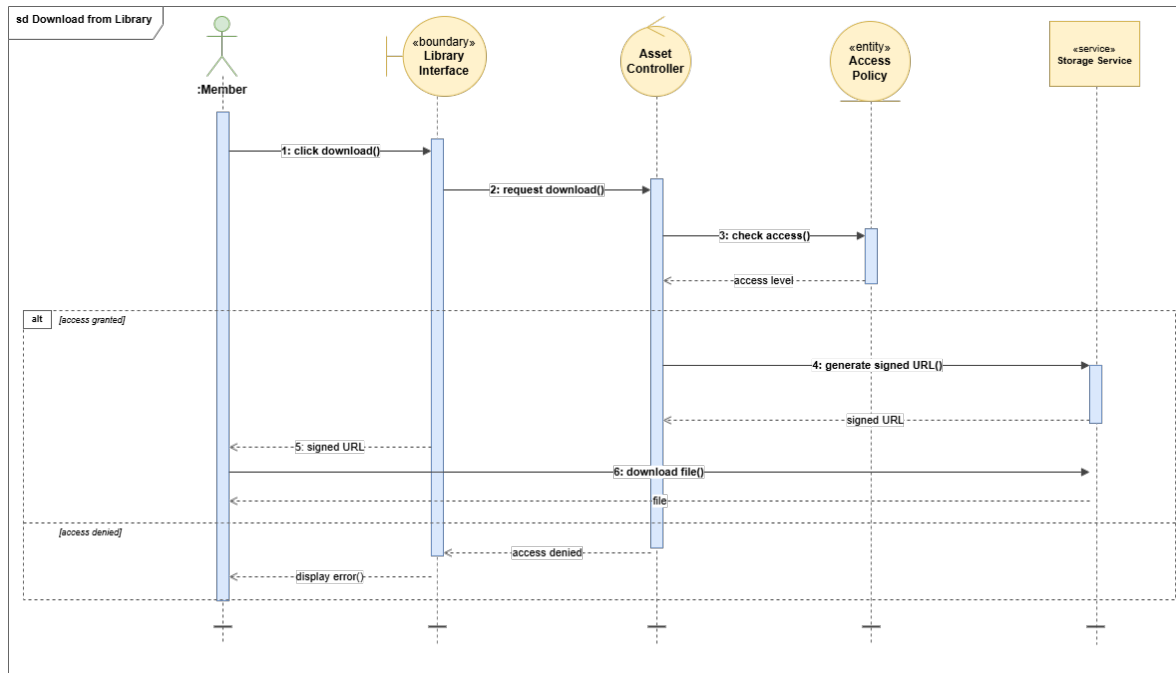


Figure 3.10: Sequence diagram «Download from Library»

Class Diagram

The class diagram of Sprint 2 introduces the **Asset** and **Category** entities alongside the purchase pipeline: **CartItem**, **Order**, **OrderItem** and **UserDownload**. Each order item locks the price and commission split at purchase time so revenue figures remain stable.

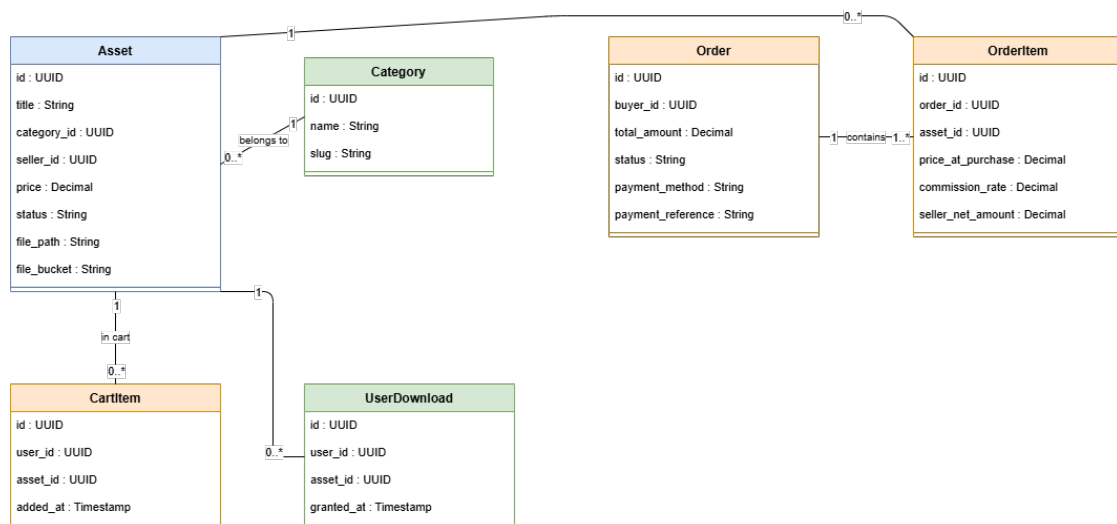


Figure 3.11: Class diagram — Sprint 2

3.2.3 Sprint 2 Realization

In this section we present the interfaces developed during Sprint 2.

«**Catalog page**». This interface lets visitors browse one of the thirteen asset categories. A sidebar exposes filters by price, rating, tags and file format. Each card shows the thumbnail, the seller, the price and an interactive preview adapted to the asset type.

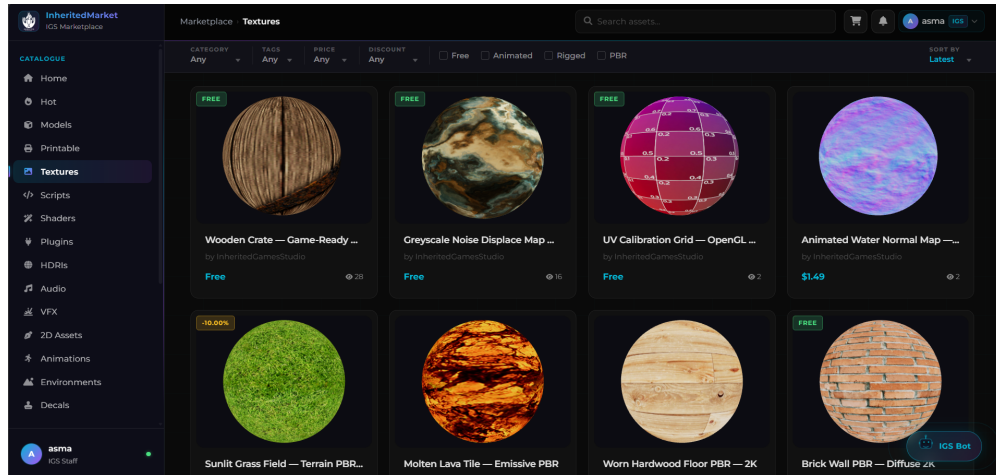


Figure 3.12: Interface «Catalog»

«**Catalog filters**». The filter sidebar lets visitors narrow results by price range, rating, tags and file format. Selections update the catalog grid instantly without a page reload.

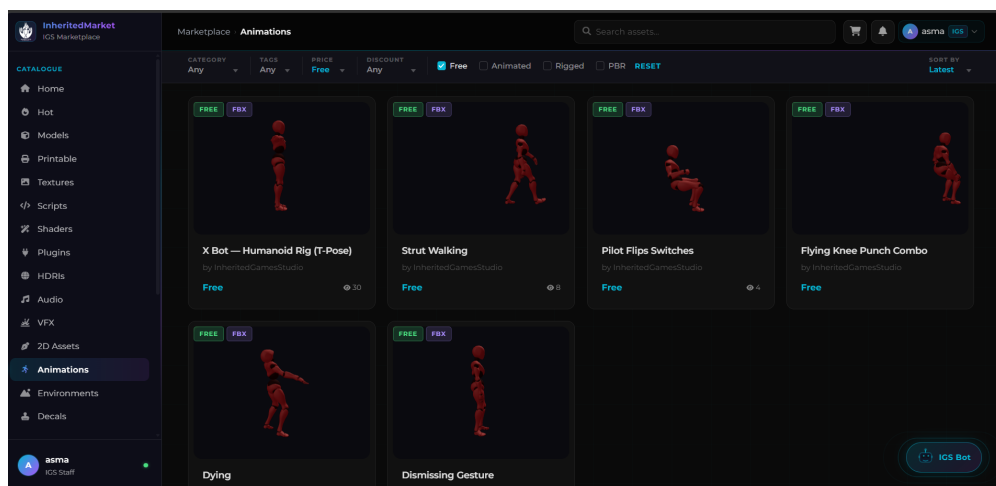


Figure 3.13: Interface «Catalog filters»

«**Shopping cart**». This interface lists the assets selected by the member. The user can remove an item or update the quantity, then proceed to checkout with Stripe or Flouci.

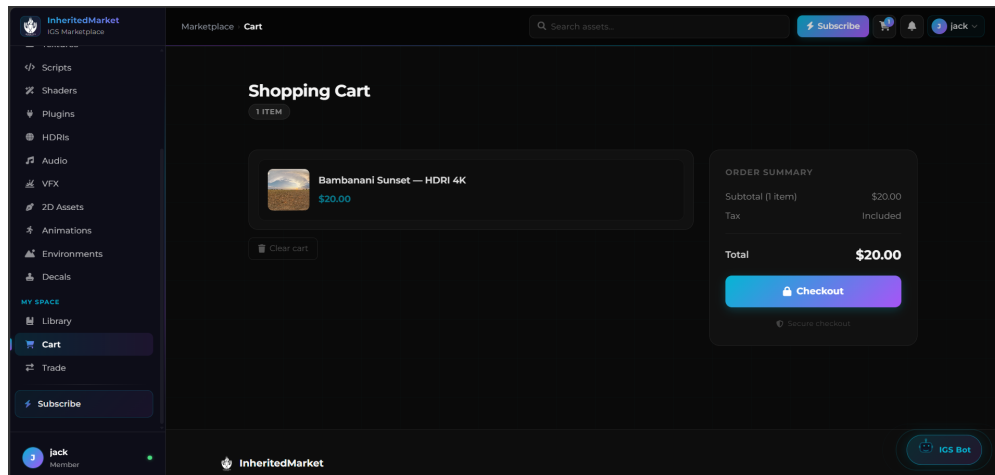


Figure 3.14: Interface «Cart»

«**Library**». This interface gathers all the assets the member has unlocked (purchased, gifted, voucher-redeemed or granted by IGS). Each row exposes a Download button that requests a one-hour signed URL from the backend.

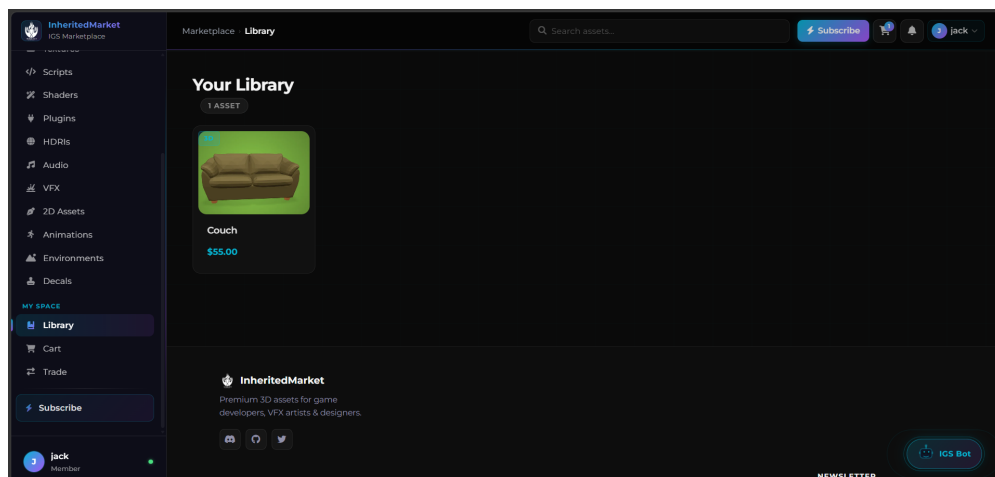


Figure 3.15: Interface «Library»

3.2.4 Sprint 2 Conclusion

At the end of Sprint 2 the public marketplace is fully functional. Visitors can browse and preview assets in 3D, members can purchase through Stripe or Flouci, and the personal library exposes time-limited signed downloads.

Conclusion

This chapter covered the authentication foundation and the public marketplace implementation across the first two sprints. The following chapter presents Sprint 3 and Sprint 4.

Seller Workflow and Workspace

Sommaire

4.1	Sprint 3: Asset Upload, Moderation and Reviews	42
4.1.1	Sprint 3 Backlog	42
4.1.2	Sprint 3 Conception	43
4.1.3	Sprint 3 Realization	47
4.1.4	Sprint 3 Conclusion	49
4.2	Sprint 4: IGS Workspace and Administration	49
4.2.1	Sprint 4 Backlog	49
4.2.2	Sprint 4 Conception	50
4.2.3	Sprint 4 Realization	56
4.2.4	Sprint 4 Conclusion	60

Introduction

This chapter presents Sprint 3 and Sprint 4, covering the seller workflow and content moderation pipeline, followed by the internal workspace for team collaboration and administration.

4.1 Sprint 3: Asset Upload, Moderation and Reviews

Sprint 3 closes the production loop of the marketplace. It covers the seller on-boarding workflow, the asset upload and moderation pipeline, the review and rating system, the peer-to-peer trade offers and the Creator Studio dashboard.

4.1.1 Sprint 3 Backlog

Table 4.1 summarizes the backlog of the third sprint.

Table 4.1: Sprint 3 Backlog

ID	User Story	Estimation
1	As a member, I want to request seller access with a justification so I can publish assets.	2 days
2	As a seller, I want to upload assets and submit them for moderation.	4 days
3	As an admin, I want a moderation queue where I can approve or reject submitted assets.	3 days
4	As a seller, I want to request edits on already-approved assets without bypassing moderation.	2 days
5	As a buyer, I want to leave a rating and a written review on owned assets.	2 days
6	As a user, I want to be notified about request, order and moderation events.	2 days
7	As a seller, I want a creator studio with stats and an asset table.	3 days
8	As a user, I want to propose peer-to-peer trades of assets I own against assets I want.	2 days

4.1.2 Sprint 3 Conception

Use Case Diagram

The figure below illustrates the use case diagram relative to Sprint 3.

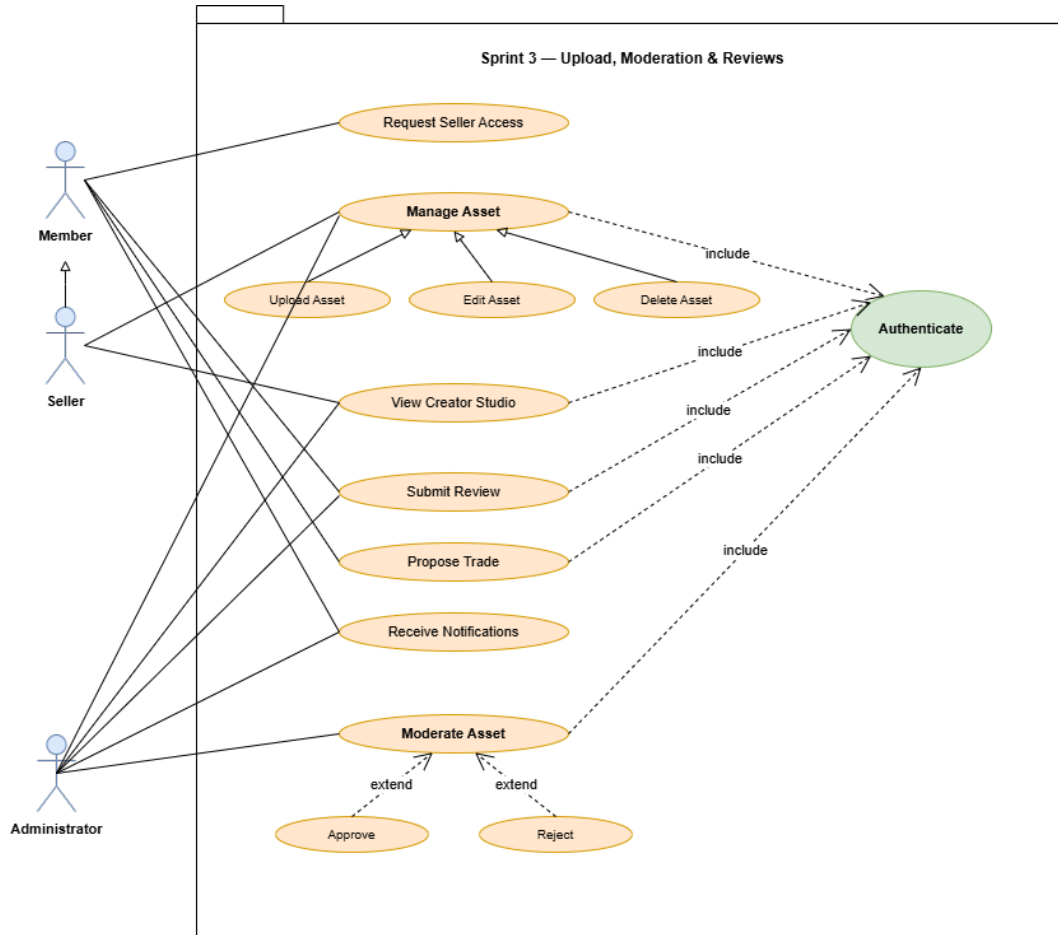


Figure 4.1: Use case diagram — Sprint 3

Table 4.2: Use case description: Upload an Asset

Use case	Upload an Asset
Actor	Seller
Pre-condition	The seller is authenticated and holds the seller role.
Post-condition	The asset is stored as pending and enters the moderation queue.
Nominal scenario	1. The seller opens the upload form. 2. The seller selects a thumbnail and an asset file. 3. The seller fills in the metadata (title, category, tags, price). 4. The system validates the file and stores it in cloud storage. 5. The asset record is created with status pending. 6. A success message is displayed.
Alternative scenario	4.1. If the file size or the MIME type is invalid, an error message is shown. 4.2. The seller selects a valid file and resumes at step 2.

Textual description of the use case «Upload an Asset».

Table 4.3: Use case description: Moderate an Asset

Use case	Moderate an Asset
Actor	Administrator
Pre-condition	A submitted asset is waiting in the moderation queue.
Post-condition	The asset is approved and published, or rejected with a note.
Nominal scenario	1. The administrator opens the moderation queue. 2. The administrator expands a pending asset. 3. The administrator inspects the file, the thumbnail and the metadata. 4. The administrator chooses Approve or Reject and adds an optional note. 5. The system updates the status, logs the action and notifies the seller.
Alternative scenario	4.1. If the administrator rejects, the seller is notified with the note and can resubmit.

Textual description of the use case «Moderate an Asset».

Table 4.4: Use case description: Submit a Review

Use case	Submit a Review
Actor	Member
Pre-condition	The member owns the asset.
Post-condition	The review and the rating are saved and visible on the asset page.
Nominal scenario	1. The member opens the detail page of an owned asset. 2. The member selects a star rating and types a written review. 3. The system verifies the ownership and stores the review. 4. The system runs sentiment analysis on the text. 5. The new review is displayed on the asset page.
Alternative scenario	3.1. If the member does not own the asset, the review form is disabled.

Textual description of the use case «Submit a Review».

Sequence Diagrams

Sequence diagram «Upload an Asset». When a seller submits the upload form, the browser sends the thumbnail and the asset file to the upload service. The system validates the MIME type at the byte level and stores each file in the correct cloud bucket. Once both files are saved, the controller creates the asset record with status pending. The seller is then notified that the submission is awaiting moderation.

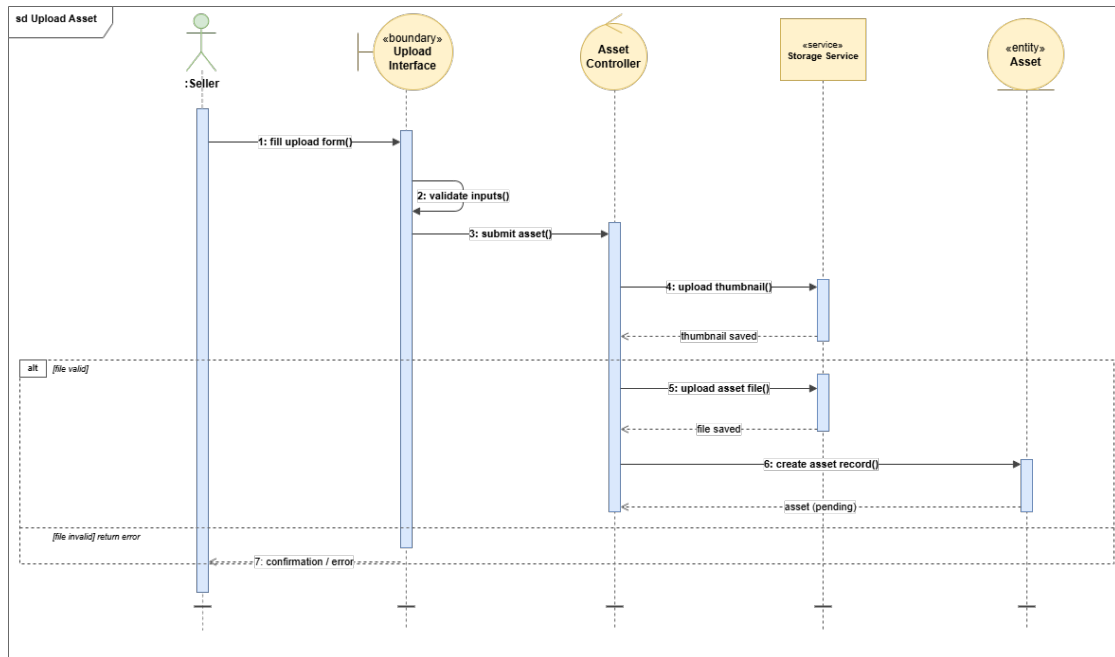


Figure 4.2: Sequence diagram «Upload an Asset»

Sequence diagram «Moderate an Asset». When an administrator opens the moderation queue, the system loads all submissions with status pending. After reviewing the file and the metadata, the administrator issues a decision. The system updates the asset status, writes an entry to the audit log and notifies the seller through the notification service.

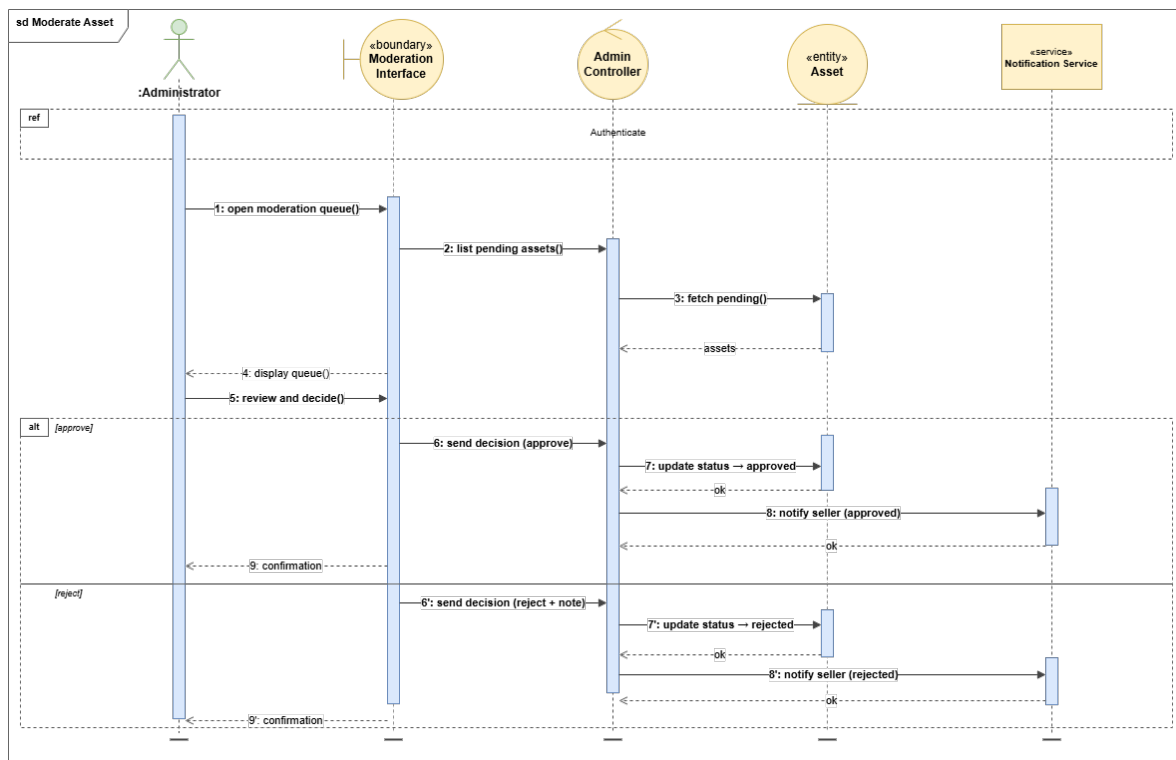


Figure 4.3: Sequence diagram «Moderate an Asset»

Class Diagram

The class diagram of Sprint 3 extends the **Asset** entity with a status state machine and connects it to **AssetEditRequest**, **Review** (with an `ai_sentiment` column filled in Sprint 5), **Trade** and **Subscription**.

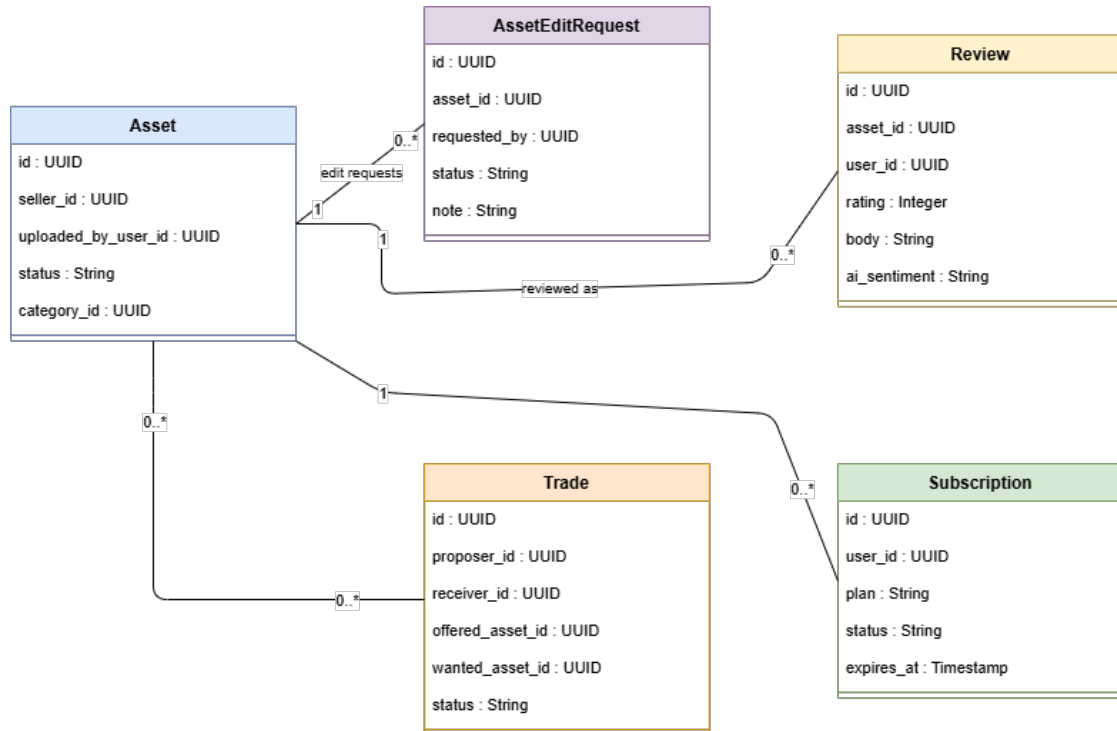


Figure 4.4: Class diagram — Sprint 3

4.1.3 Sprint 3 Realization

In this section we present the interfaces developed during Sprint 3.

«**Seller profile**». This interface gathers the public information of a seller: avatar, biography, published assets, total sales and aggregated rating. Visitors can open it from any asset card.

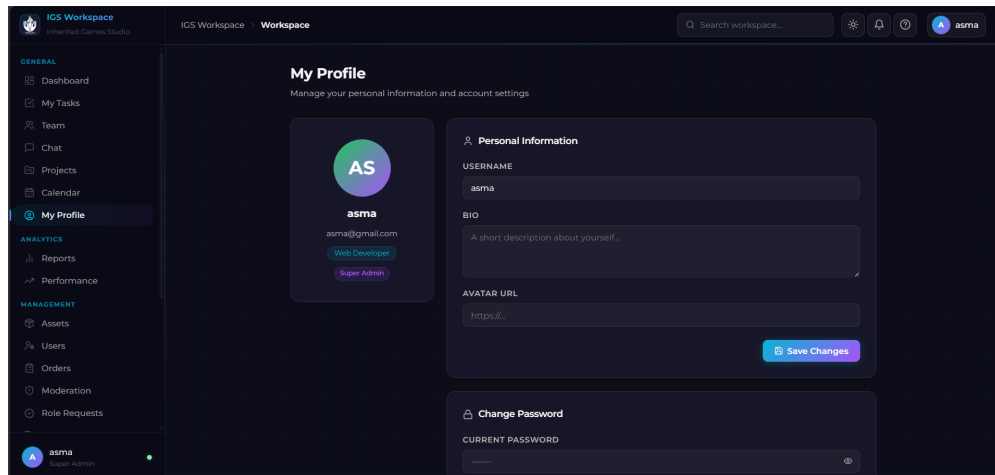


Figure 4.5: Interface «Seller profile»

«**Asset reviews**». This interface shows the chronological list of written reviews and ratings for an asset. Each card displays the rating, the author, the body, and a sentiment badge produced by the AI service introduced in Sprint 5.



Figure 4.6: Interface «Reviews»

«**Creator Studio**». This interface gives sellers a personal performance dashboard: total revenue, active listings, recent sales and a paginated asset table with per-row statistics.

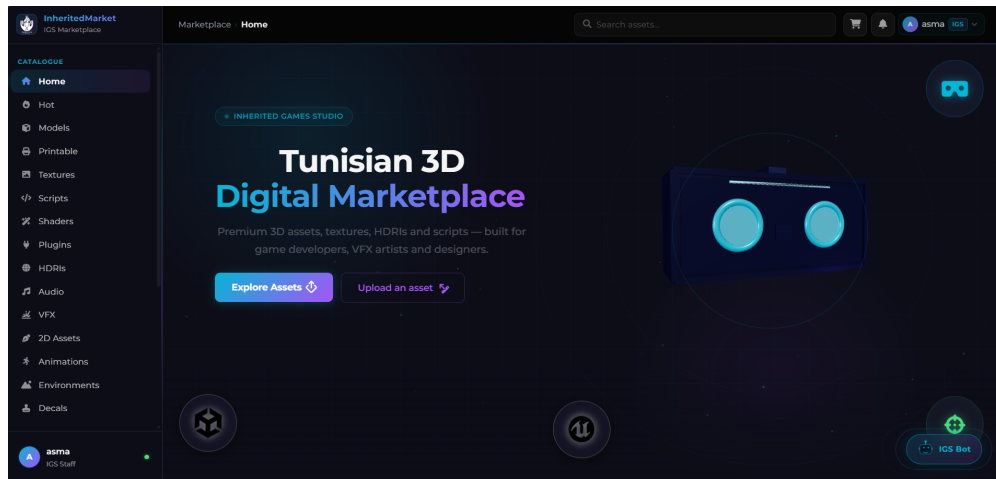


Figure 4.7: Interface «Creator Studio»

«**Moderation queue**». This interface lets administrators consult every pending submission. Each row is expandable to show the asset metadata, the file preview, the seller note and the decision buttons.

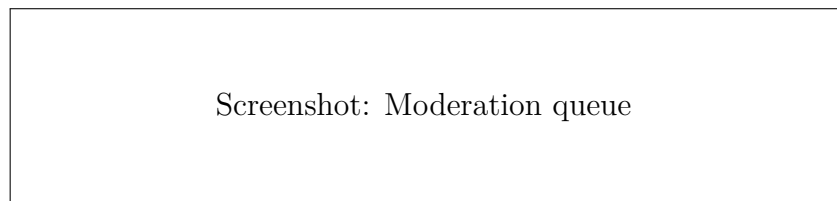


Figure 4.8: Interface «Moderation queue»

4.1.4 Sprint 3 Conclusion

Sprint 3 closes the production loop. Sellers can publish assets, administrators control the catalog through moderation, buyers can rate and review, and notifications keep every actor informed of state changes.

4.2 Sprint 4: IGS Workspace and Administration

Sprint 4 delivers the internal IGS Workspace. It covers project and task management, the team chat, the calendar, the moderation of intern uploads and intern access requests, the administrative panel for users and categories, and the system pages reserved for super-administrators.

4.2.1 Sprint 4 Backlog

Table 4.5 summarizes the backlog of the fourth sprint.

Table 4.5: Sprint 4 Backlog

ID	User Story	Estimation
1	As an internal user, I want a workspace with role-gated navigation.	3 days
2	As an intern, I want a Kanban task board with priorities and assignees.	3 days
3	As an internal user, I want to create and manage projects with members and a date range.	2 days
4	As an internal user, I want a calendar that shows task deadlines and project milestones.	1 day
5	As an internal user, I want a real-time chat.	3 days
6	As an employee, I want analytics reports on revenue, sales and per-team performance.	2 days
7	As an admin, I want to manage users, role requests, categories and vouchers.	3 days
8	As an admin, I want to approve or reject intern asset access requests.	2 days
9	As a super-admin, I want to consult audit logs and configure the platform.	2 days

4.2.2 Sprint 4 Conception

Use Case Diagram

The figure below illustrates the use case diagram relative to Sprint 4.

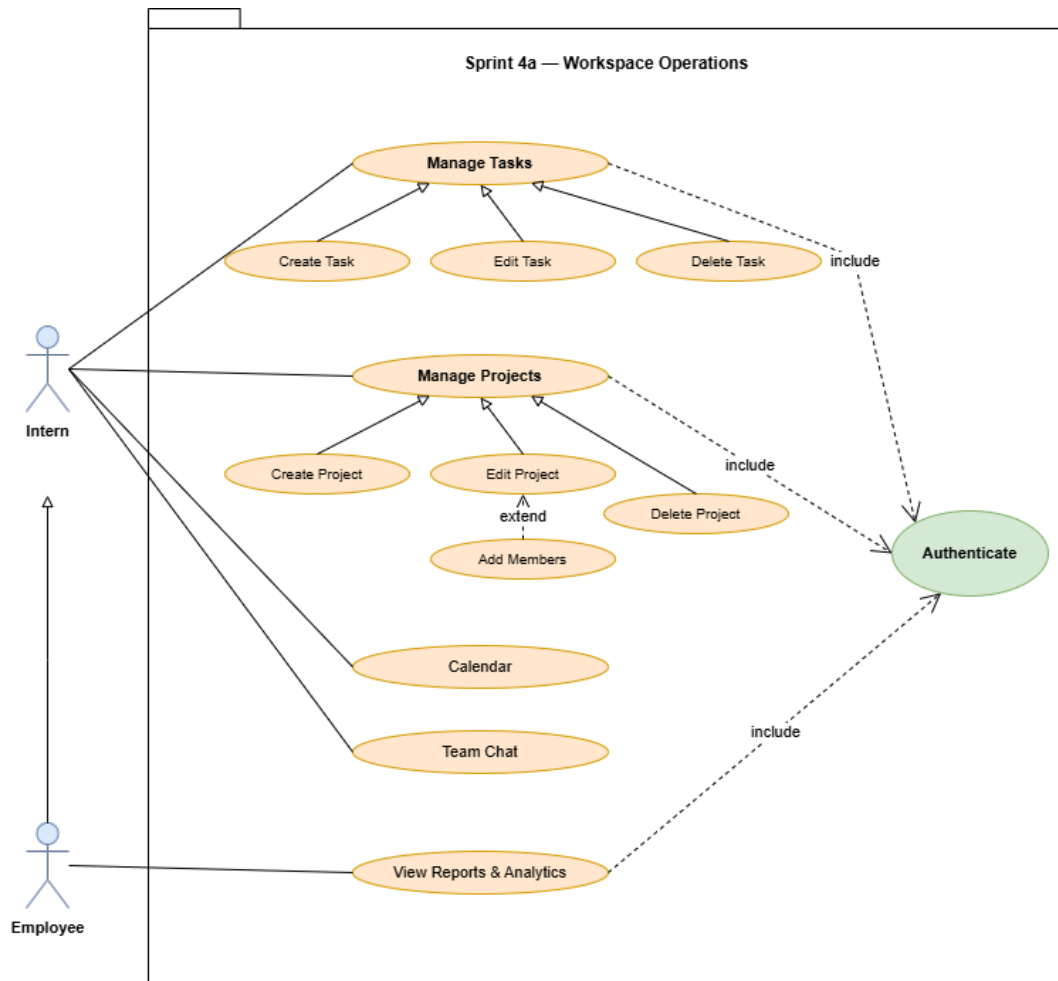


Figure 4.9: Use case diagram — Sprint 4 (Part A)

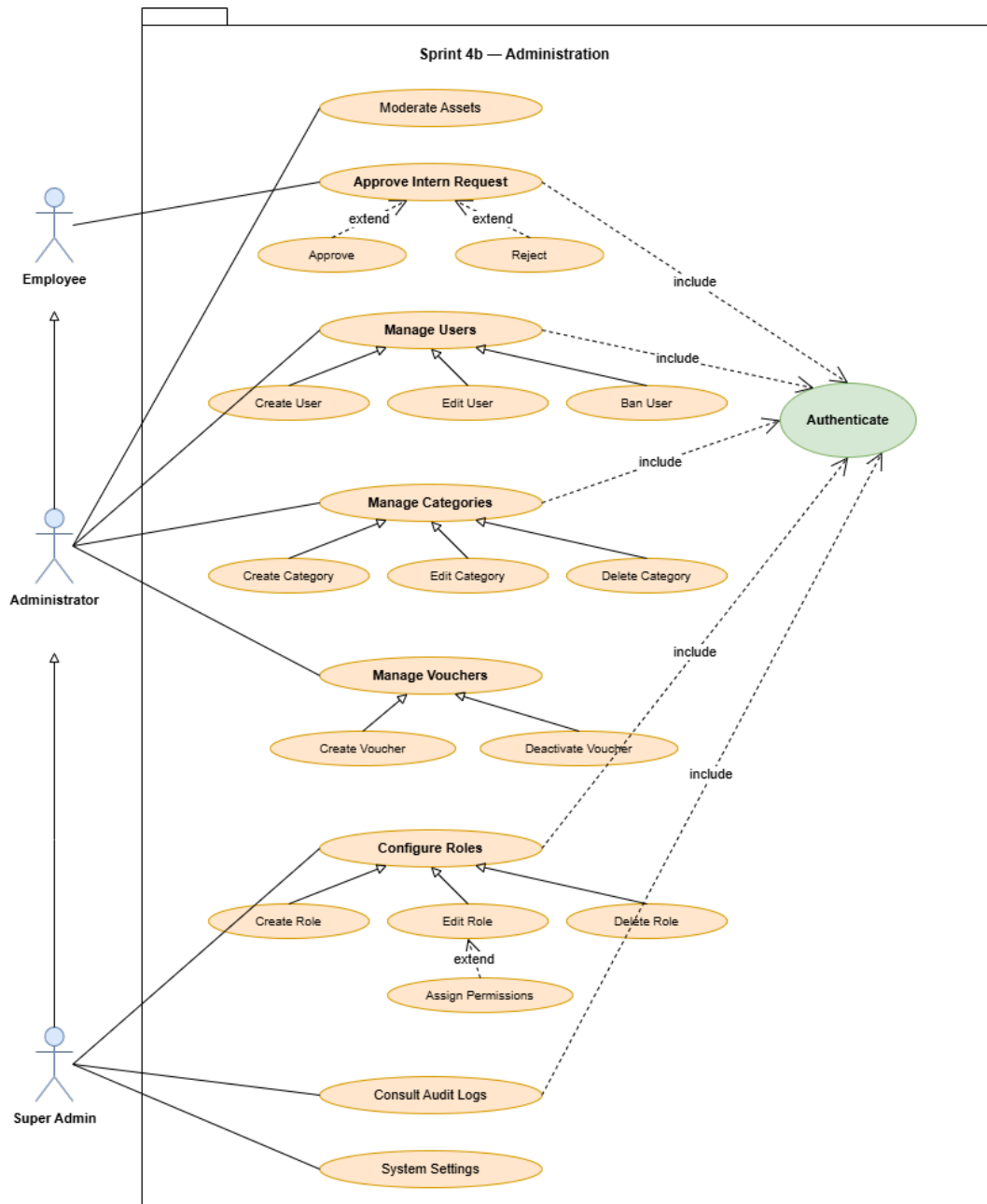


Figure 4.10: Use case diagram — Sprint 4 (Part B)

Table 4.6: Use case description: Create a Task

Use case	Create a Task
Actor	Internal user (Intern, Employee, Admin)
Pre-condition	The user is authenticated in the workspace.
Post-condition	The task is created on the Kanban board and the assignees are notified.
Nominal scenario	1. The user opens the project board. 2. The user clicks New Task. 3. The user fills in the title, the description, the priority and the assignees. 4. The system creates the task with status To-do. 5. The system notifies the assignees through Socket.IO. 6. The task appears on the Kanban board.
Alternative scenario	3.1. If the user is not a member of the project, an error message is shown.

Textual description of the use case «Create a Task».

Table 4.7: Use case description: Approve Intern Access Request

Use case	Approve Intern Access Request
Actor	Employee / Administrator
Pre-condition	An intern submitted a request for a paid IGS asset.
Post-condition	The request is approved and the asset is added to the intern's library, or rejected with a note.
Nominal scenario	1. The employee opens the workspace moderation page. 2. The employee selects a pending intern request. 3. The employee chooses Approve or Reject and adds a note. 4. The system updates the request status. 5. On approval, the asset is added to the intern's library. 6. The intern is notified of the decision.
Alternative scenario	3.1. If the employee rejects, the intern is notified and may submit a new request.

Textual description of the use case «Approve Intern Access Request».

Table 4.8: Use case description: Consult Audit Logs

Use case	Consult Audit Logs
Actor	Super-administrator
Pre-condition	The super-administrator is authenticated in the workspace.
Post-condition	The filtered list of audit entries is displayed.
Nominal scenario	1. The super-administrator opens the audit log page. 2. The super-administrator applies filters on action type or user. 3. The system queries the audit log table. 4. The system returns a paginated list of entries. 5. The super-administrator expands an entry to inspect the JSON metadata.
Alternative scenario	4.1. If no entry matches the filter, an empty-state message is displayed.

Textual description of the use case «Consult Audit Logs».

Sequence Diagrams

Sequence diagram «Create a Task». When an internal user submits a new task, the browser sends the form to the workspace controller. The controller verifies the project membership, inserts the task in the database with status To-do and writes a notification row for each assignee. The system then emits a Socket.IO event so the assignees see the task appear in real time on their Kanban board.

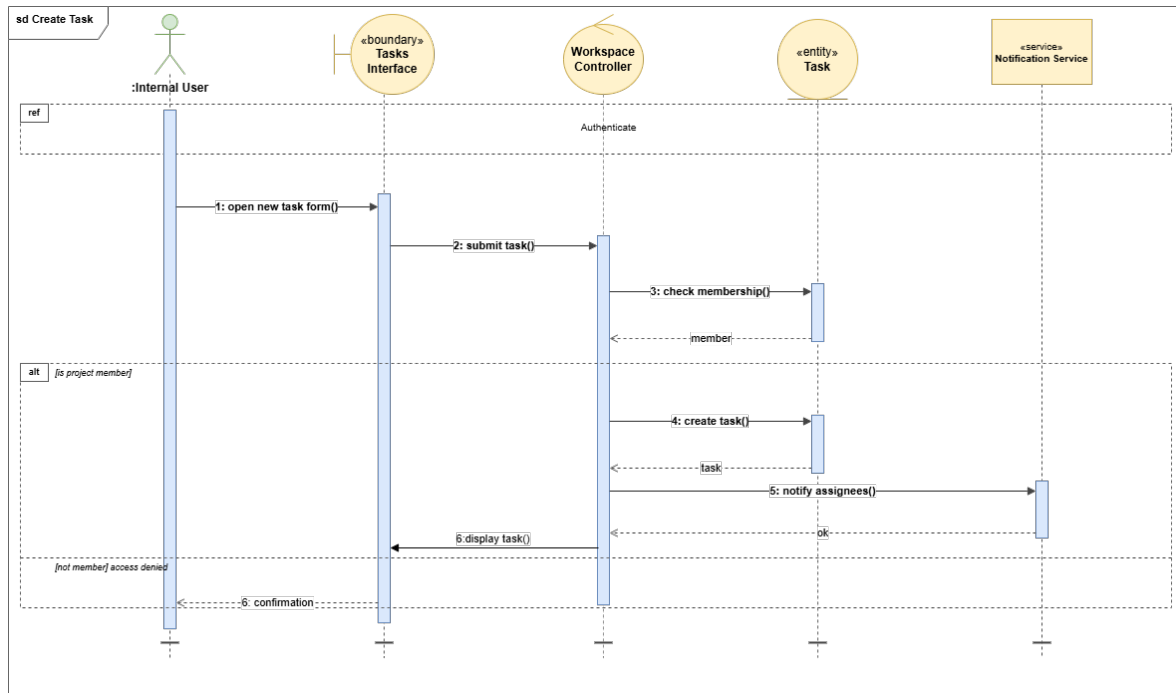


Figure 4.11: Sequence diagram «Create a Task»

Sequence diagram «Approve Intern Access Request». When an intern requests access to a paid IGS asset, the system stores the request and notifies the supervisors. When the supervisor opens the workspace moderation page and approves the request, the system adds the asset to the intern's library and sends a notification with the decision back to the intern.

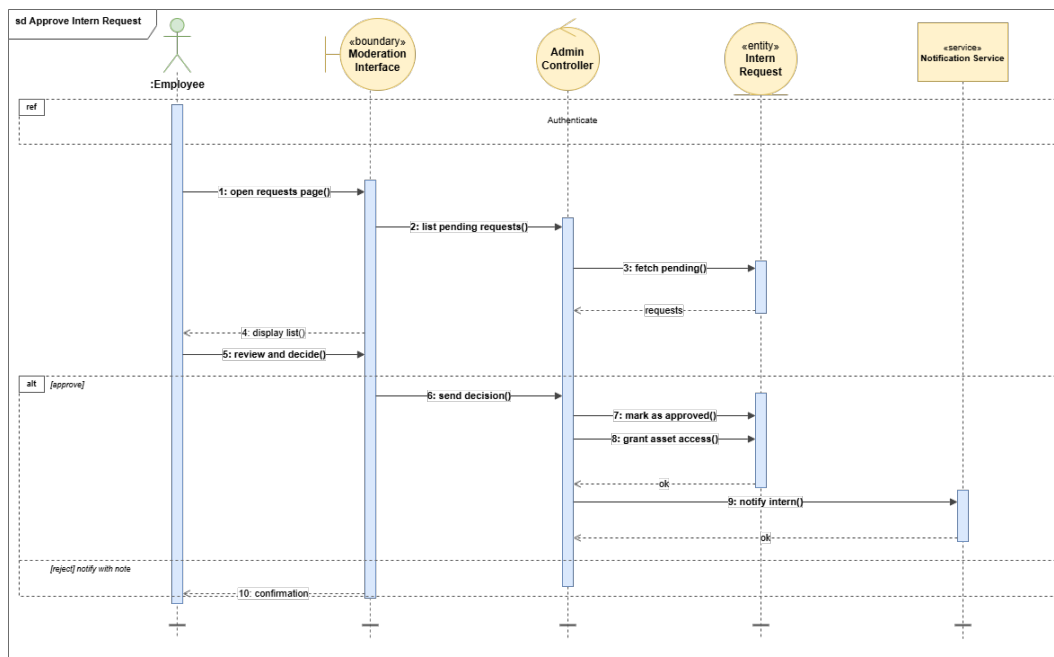


Figure 4.12: Sequence diagram «Approve Intern Access Request»

Class Diagram

The class diagram of Sprint 4 covers the internal collaboration layer: **Project** and **WorkspaceTask** for project tracking, **ChatChannel** and **Message** for real-time communication, **InternAssetRequest** for supervised access and **Voucher** for the discount code management.

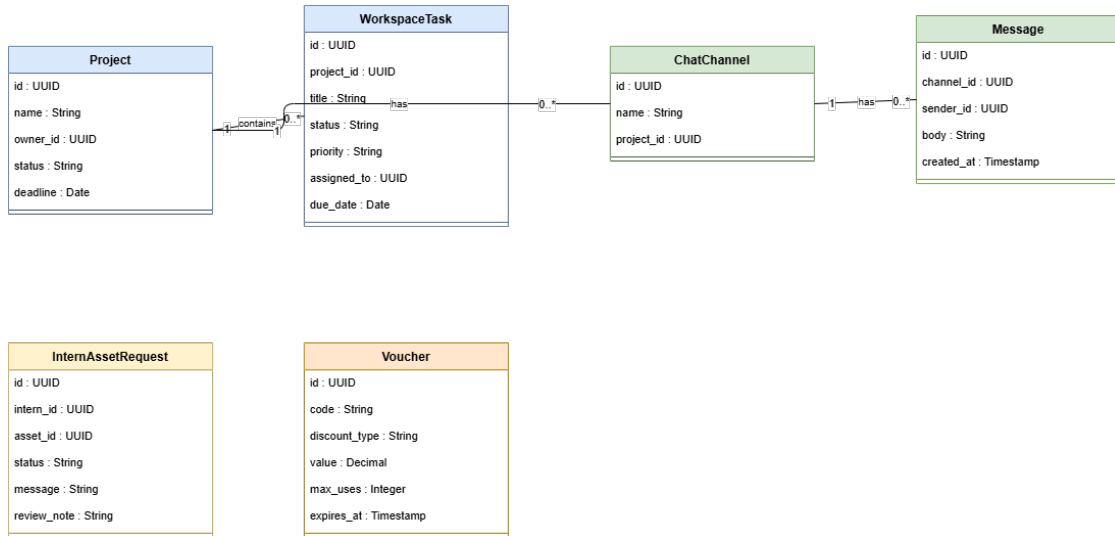


Figure 4.13: Class diagram — Sprint 4

4.2.3 Sprint 4 Realization

In this section we present the interfaces developed during Sprint 4.

«**Workspace dashboard**». This interface is the landing page of the workspace. It centralises the team's recent activity: open tasks, ongoing projects, pinned notifications and quick access to the chat.

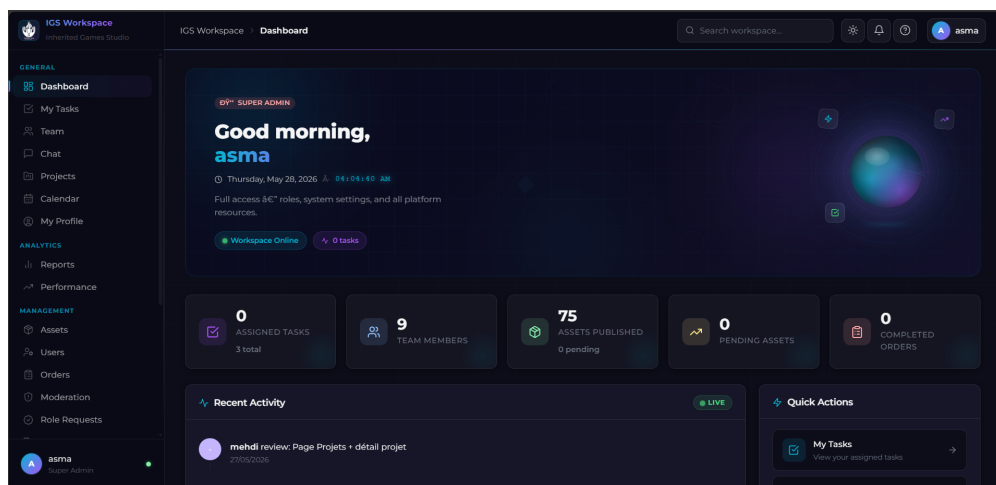


Figure 4.14: Interface «Workspace dashboard»

«**Kanban task board**». This interface displays tasks organised in four columns (To-do, In progress, Review, Done). Each card carries a priority badge, an assignee avatar and a due date.

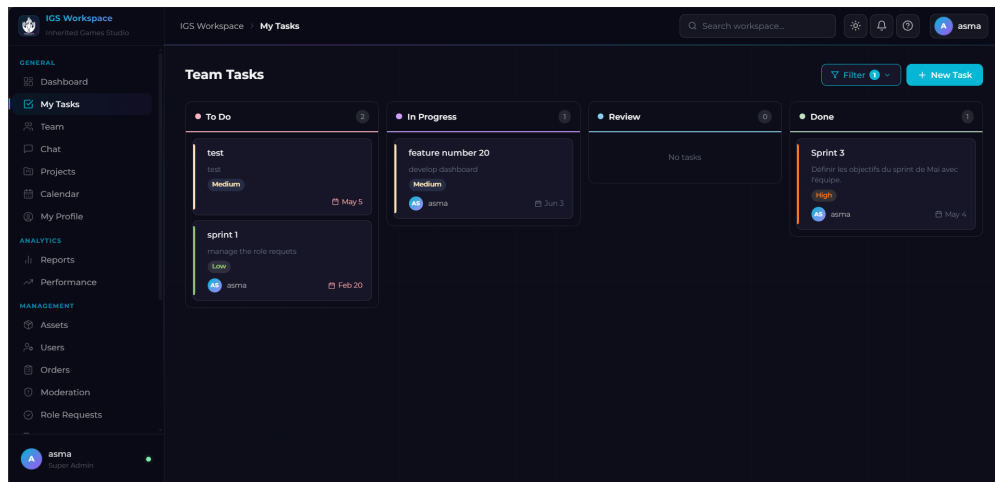


Figure 4.15: Interface «Tasks»

«**User management**». This interface lists every platform user. The administrator can change the role, edit the profile or ban an account.

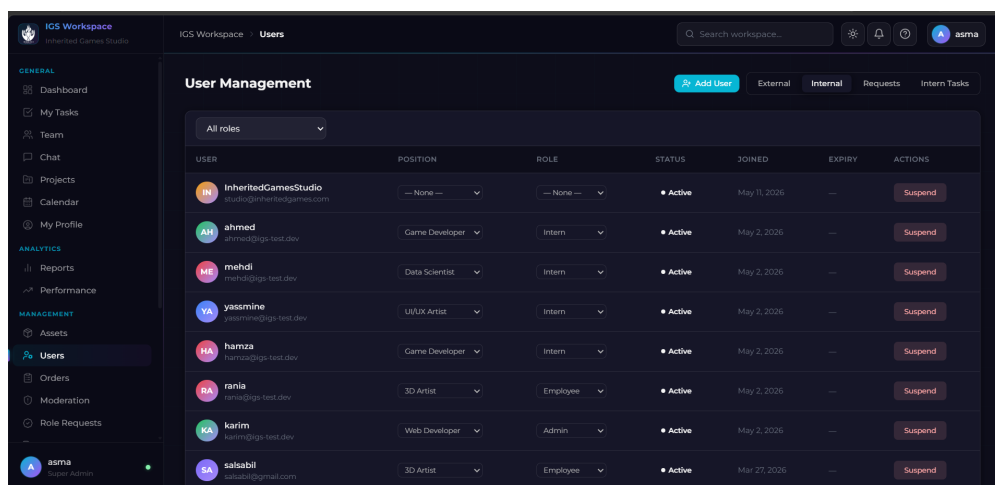


Figure 4.16: Interface «User management»

«**Order management**». This interface displays every order placed on the platform with the payment status, the buyer, the amount and the commission. The administrator can refund an order or inspect the line items.

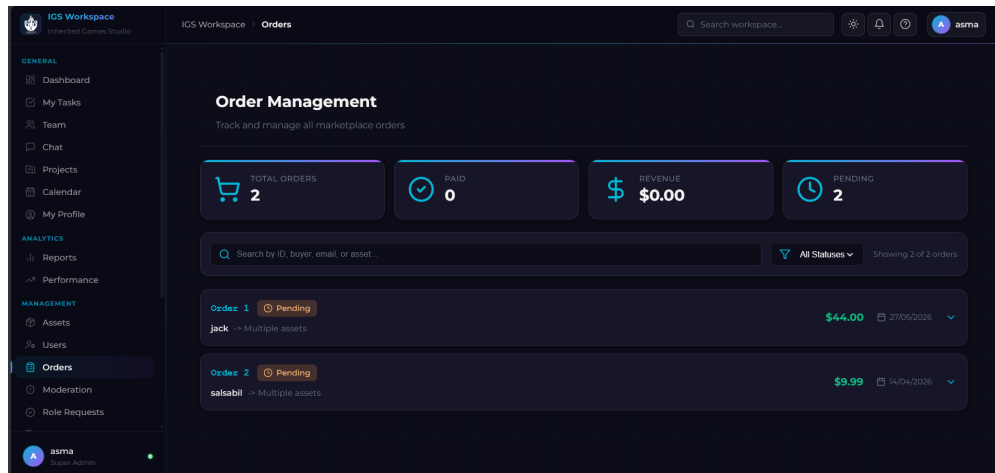


Figure 4.17: Interface «Order management»

«**Roles and permissions**». This interface lists every role with its current permission set and the live user count. Super-administrators can edit the role-permission assignment.

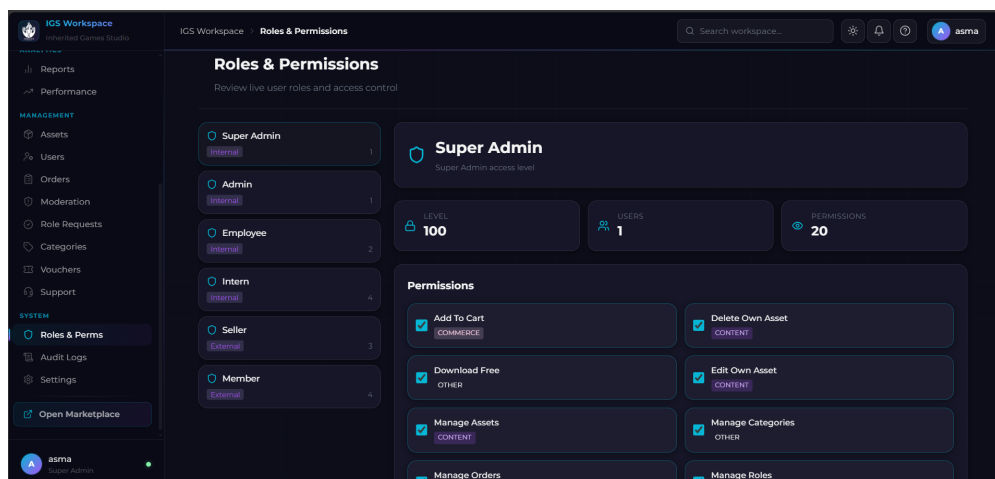


Figure 4.18: Interface «Roles and permissions»

«**Audit logs**». This interface lets the super-administrator filter the audit log by action and by user, browse paginated results and expand any entry to inspect its JSON metadata.

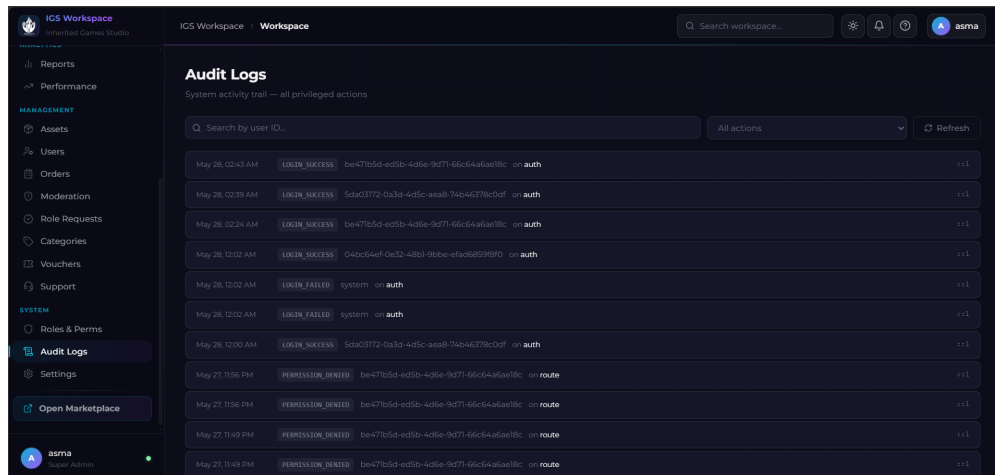


Figure 4.19: Interface «Audit logs»

«**System settings**». This interface gathers the platform-wide configuration. The most critical entry is the commission rate applied at order time.

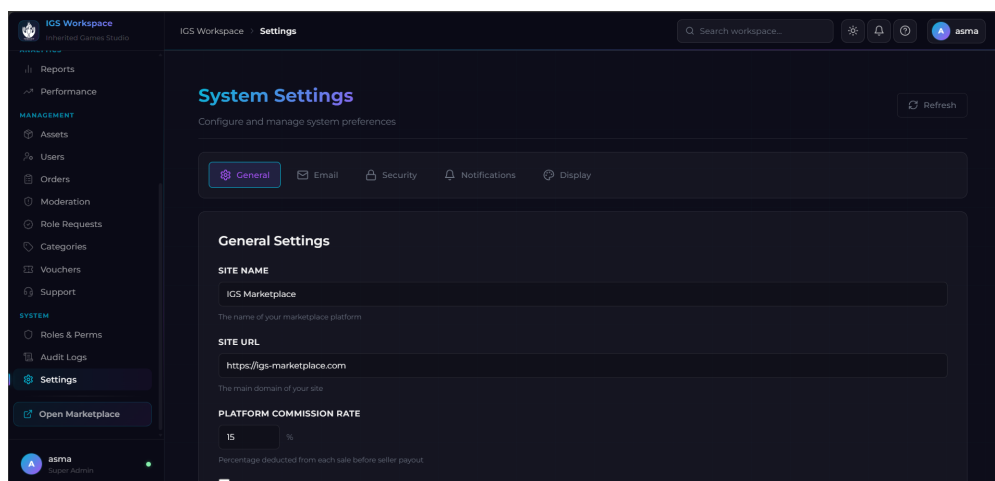


Figure 4.20: Interface «System settings»

«**Support inbox**». This interface centralises the support tickets opened by sellers. The administrator can filter by status, send a message and close the ticket.

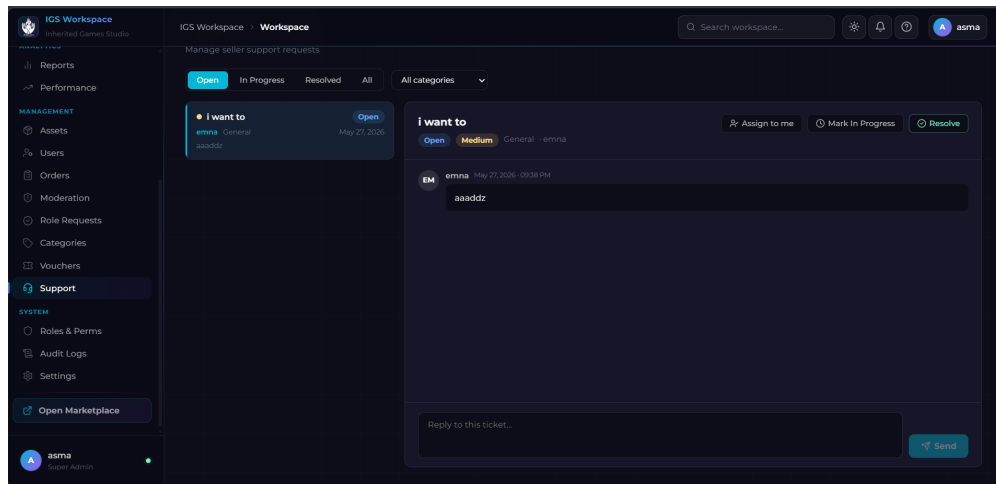


Figure 4.21: Interface «Support inbox»

4.2.4 Sprint 4 Conclusion

At the end of Sprint 4 the studio team has a fully functional internal ERP. Daily operations are centralised, moderators have a single queue for external and intern submissions, and super-administrators control the platform from a dedicated system menu while every sensitive action is traced through the audit log.

Conclusion

This chapter covered the seller production workflow and the internal workspace dashboard across Sprint 3 and Sprint 4. The following chapter presents the advanced features of Sprint 5.

Advanced Features

Sommaire

5.1	Sprint 5 Backlog	62
5.2	AI Services	62
5.2.1	AI Chatbot	63
5.2.2	Review Sentiment Analysis	66
5.2.3	Personalized Recommendations	68
5.3	Security Hardening	69
5.4	VR/AR Immersive Preview	70
5.4.1	Analysis	70
5.4.2	Realization	71
5.5	Deployment on Hostinger VPS	71
5.5.1	Target Infrastructure	72
5.5.2	Key Deployment Concepts	72
5.5.3	CI/CD Pipeline	72

Introduction

This chapter presents Sprint 5, covering the AI services integrated into the platform, the immersive VR/AR preview feature, and the production deployment infrastructure.

5.1 Sprint 5 Backlog

Table 5.1: Sprint 5 Backlog

ID	User Story	Estimation
1	As a user, I want a chatbot widget available on every page so I can ask marketplace questions.	3 days
2	As a user, I want the chatbot to recognize my intent and reply accordingly.	2 days
3	As a user, I want my submitted review classified automatically as positive, neutral or negative.	2 days
4	As a seller, I want an aggregate sentiment summary of all reviews on my assets.	1 day
5	As a user, I want personalized asset recommendations based on my activity.	3 days
6	As a guest, I want to see trending assets when no personalization data is available.	1 day
7	As an admin, I want the API protected against brute-force with per-route rate limits.	1 day
8	As an admin, I want logout tokens to be effectively revoked, not just expired.	1 day
9	As an admin, I want uploaded images validated at the byte level to prevent MIME-confusion attacks.	1 day
10	As a maintainer, I want the platform deployed on Hostinger VPS with Nginx and PM2.	2 days

5.2 AI Services

Three AI services enrich the platform with intelligent behaviour. Each was selected to match the available infrastructure and the nature of the data: a large language model

for conversational assistance, a pre-trained transformer for sentiment classification, and a deterministic similarity algorithm for personalized recommendations.

5.2.1 AI Chatbot

Analysis

Overview. The chatbot is a conversational assistant integrated into every page of the marketplace. It allows users to search the catalog, ask platform-related questions and retrieve order information in natural language. The system operates through a three-stage pipeline: intent classification, optional catalog data retrieval, and response generation. This architecture keeps latency low by routing simple requests locally and only invoking the language model when a free-form answer is required.

Stage 1 — Intent Classification with TF-IDF and Linear SVM. Intent classification determines the purpose of a user message before any further processing. The platform uses a two-step approach: statistical vectorization followed by a linear classifier.

TF-IDF — Term Frequency–Inverse Document Frequency

TF-IDF [37] is a statistical measure that quantifies the importance of each word in a message relative to the entire training corpus. Words that appear frequently in the message but rarely across all messages receive high scores, making them strong discriminators. Formally, for a word w in document d from corpus D :

$$\text{TF-IDF}(w, d, D) = \frac{f_{w,d}}{\sum_{w' \in d} f_{w',d}} \times \log\left(\frac{|D|}{|\{d' \in D : w \in d'\}|}\right)$$

Each message is transformed into a sparse numerical vector representing its lexical content.

Support Vector Machine

A **Support Vector Machine** [38] then classifies the TF-IDF vector. SVMs find the optimal separating hyperplane between training classes by maximizing the margin between the closest examples of each class (the *support vectors*). The linear variant is particularly effective for text classification tasks, since TF-IDF feature spaces are generally linearly separable and the linear kernel trains efficiently on high-dimensional sparse data [39].

The classifier recognizes six intent categories, illustrated in Table 5.2.

Table 5.2: Intent categories handled by the classifier

Intent	Example utterance
Product search	“Show me animated character models under \$10”
Platform help	“How do I upload my first asset?”
Order status	“Where is my download?”
Seller information	“How do commissions work?”
Greeting	“Hello, what can you help me with?”
Out of scope	“What is the weather today?”

Stage 2 — Retrieval-Augmented Generation. When a product-search intent is detected, the platform queries the asset catalog using keywords from the message and injects the matching results into the model’s context window. This **retrieval-augmented generation** (RAG) pattern [42] grounds the model’s response in real catalog data and eliminates the risk of the model fabricating non-existent products.

Stage 3 — Response Generation with a Large Language Model. The final response is generated by **Qwen2.5-72B-Instruct** [41], a large language model developed by the Qwen Team at Alibaba Cloud.

Transformer Architecture and Large Language Models

Large language models are built on the **Transformer** architecture [40], which relies on a *self-attention mechanism* allowing every token in the input sequence to attend to every other token simultaneously, capturing long-range dependencies without recurrence. Pre-trained on massive text corpora and subsequently fine-tuned with instruction-following data, these models generate fluent, contextually coherent answers conditioned on a provided system prompt and conversation history.

The model is served through the Hugging Face Inference API [28], eliminating the need for local GPU infrastructure. A rule-based fallback is activated whenever the external service is unreachable, ensuring the assistant remains functional at all times.

Sequence diagram «AI Chatbot». The figure below illustrates the three-stage pipeline executed on each user message.

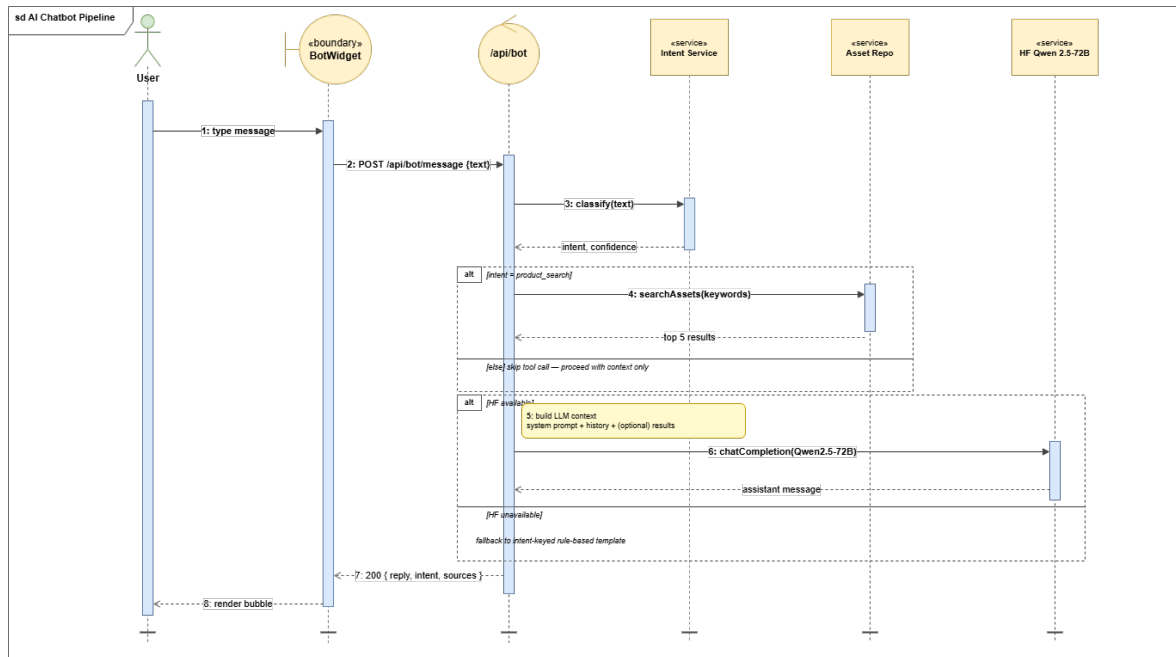


Figure 5.1: Sequence diagram «AI Chatbot»

Realization

The chatbot is available as a floating widget on every page of the marketplace, accessible to both authenticated users and guests. A submitted message passes through the three-stage pipeline and the generated answer is returned to the interface in real time.

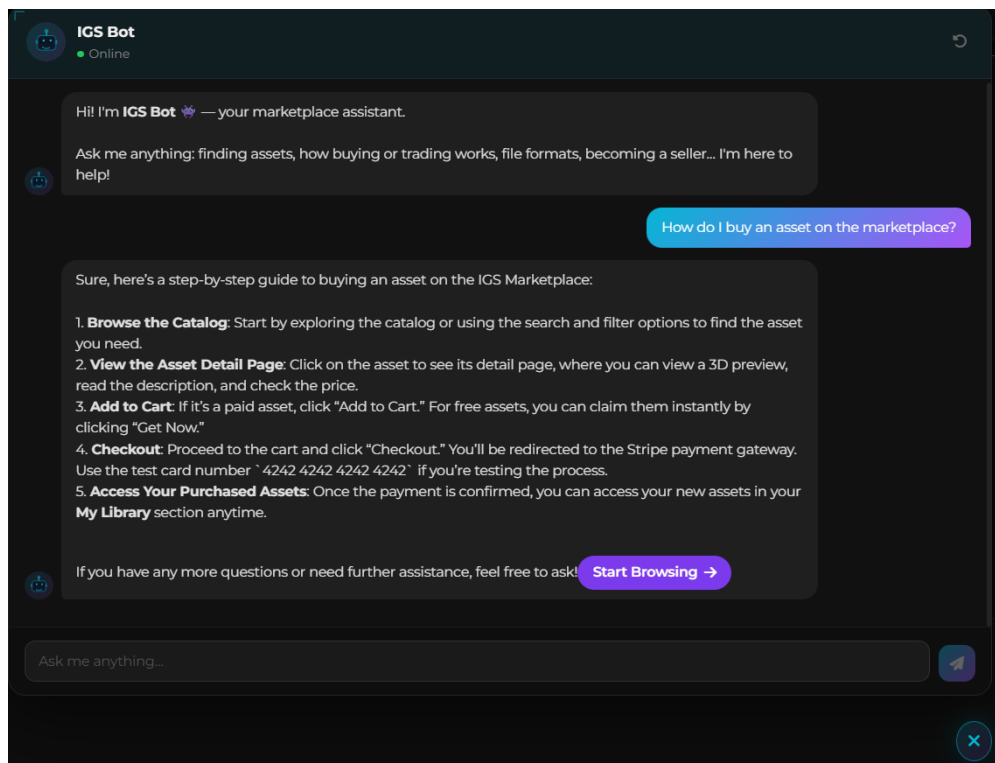


Figure 5.2: AI chatbot widget — intent-aware assistant available on every page

Conclusion

The combination of a local intent classifier and a cloud language model achieves both responsiveness and generative quality. Straightforward intents are handled immediately, while complex natural-language queries benefit from the reasoning capacity of the large model.

5.2.2 Review Sentiment Analysis

Analysis

Overview. Every review submitted by a buyer is automatically analyzed to determine whether its content is positive, neutral or negative. This sentiment classification supports moderators in auditing large volumes of user feedback and provides sellers with an aggregated view of how their assets are perceived, without requiring manual reading of each entry.

Primary Model — DistilBERT. The primary classification engine is **DistilBERT** [44], a compressed variant of the **BERT** architecture.

BERT — Bidirectional Encoder Representations from Transformers

BERT [43] is a pre-trained deep bidirectional Transformer that reads the entire input sequence simultaneously, capturing context from both sides of each word. This bidirectional attention mechanism makes it significantly more effective at understanding negation and nuance than models that process text strictly left to right.

Knowledge Distillation and DistilBERT

DistilBERT [44] is obtained through **knowledge distillation**: a smaller *student* model is trained to replicate the output distribution of a larger *teacher* model (BERT), rather than learning from raw labels alone. The result retains approximately 97 % of BERT’s performance on downstream tasks while using 40 % fewer parameters and running 60 % faster.

The variant used in this project was fine-tuned on the **Stanford Sentiment Treebank** (SST-2) [45], a benchmark dataset of sentence-level sentiment annotations drawn from movie reviews. Its classification of short evaluative sentences transfers naturally to marketplace reviews. The model returns a binary label (positive or negative) alongside a confidence score; reviews with a confidence below 0.65 are mapped to the *neutral* class to avoid committing to a label when the signal is ambiguous.

Fallback Model — Logistic Regression. A secondary **Logistic Regression** [39] classifier trained on TF-IDF features serves as an offline fallback when the cloud inference API is unavailable. Logistic Regression estimates the probability of class membership by applying the logistic function to a linear combination of input features. It produces a three-class output (positive, neutral, negative) and is well suited to this role because of its low computational footprint and interpretable decision boundaries. Table 5.3 summarizes the runtime decision logic.

Table 5.3: Sentiment classification — decision logic

Condition	Model	Output label
Cloud API available, confidence ≥ 0.65	DistilBERT	positive or negative
Cloud API available, confidence < 0.65	DistilBERT	neutral
Cloud API unavailable	Logistic Regression	positive, neutral or negative

Realization

Each review is analyzed immediately upon submission. The resulting sentiment label is stored alongside the review and displayed as a color-coded badge on the asset detail page. The seller dashboard aggregates all labels into a summary chart visible from the Creator Studio.

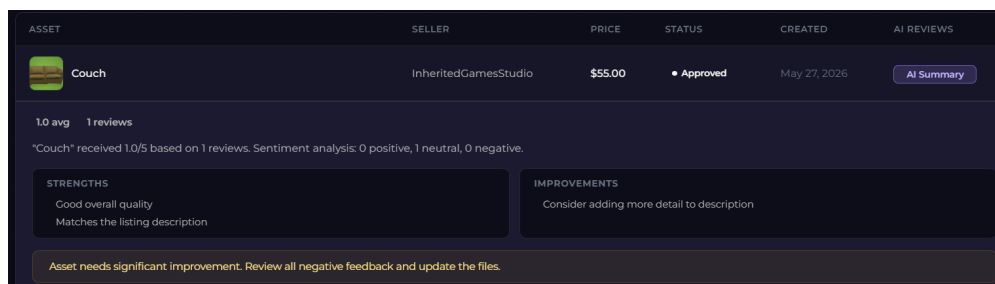


Figure 5.3: Sentiment analysis — review badges and seller aggregate sentiment summary

Conclusion

Automated sentiment classification reduces the effort required to audit large volumes of reviews. Moderators can quickly identify trends and sellers receive actionable feedback without reading every individual entry.

5.2.3 Personalized Recommendations

Analysis

Overview. The recommendation system surfaces relevant assets to each user based on their interaction history. It follows a **content-based filtering** approach [46], which derives user preferences from past behaviour and matches them against asset attributes rather than comparing users with one another. This design does not require a large user base to produce meaningful results, making it well-suited to a growing marketplace.

Similarity Measure — Jaccard Index. Each asset is represented by its set of descriptive tags.

Jaccard Similarity Index

The **Jaccard similarity index** [47] quantifies the overlap between two sets A and B as the ratio of their intersection to their union:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

A value of 1 indicates perfectly identical sets; a value of 0 indicates no overlap at all. The Jaccard index is well suited to recommendation tasks based on tags because the attributes are binary (present or absent) and the sets vary widely in size across categories.

User Profile Construction. The user’s preference profile is built from four interaction types, each weighted according to the strength of preference it expresses:

Table 5.4: User interaction signals and their weights

Interaction	Weight	Rationale
Asset like	5	Strongest explicit preference signal
Paid purchase	4	High-confidence revealed preference
Asset download	3	Direct acquisition of content
Star rating	1–5	Weighted proportionally to the numeric value

Diversity and Cold-Start Handling. To prevent the ranked list from being dominated by near-identical assets, a post-processing diversity pass demotes any item whose

tag overlap with a higher-ranked result exceeds 70%. Users with no recorded interactions — guests and newly registered accounts — receive a trending list ranked by recent download and purchase frequency, and the interface displays “Trending Now” rather than “Recommended for You” in this cold-start state.

Realization

The recommendation page is accessible to both authenticated and anonymous users. Users with sufficient interaction history receive a personalized ranking; all others see the trending catalog. The interface is visually consistent with the standard catalog pages.

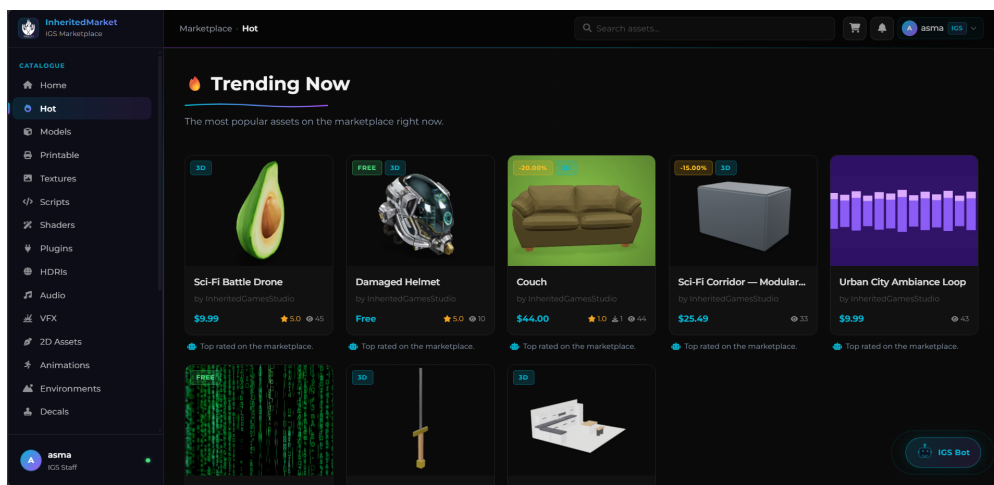


Figure 5.4: Recommendations page — personalized (warm user) and trending (cold start)

Conclusion

Content-based filtering with Jaccard similarity delivers personalized results without requiring collaborative data or model retraining. The cold-start strategy ensures every visitor receives meaningful content from the first interaction.

5.3 Security Hardening

Three independent security mechanisms protect the platform against the most common attack vectors catalogued in the OWASP Top Ten [33].

- **Rate limiting.** Each sensitive endpoint is protected by a per-IP request ceiling. Authentication attempts are limited to ten requests per fifteen-minute window, and account registration is capped at five per hour. This mitigates brute-force and credential-stuffing attacks by making exhaustive enumeration computationally impractical.

- **Token revocation.** Authentication tokens are immediately invalidated upon logout through a server-side blacklist consulted on every authenticated request. This ensures that a token intercepted after the user has signed out cannot be reused, regardless of its expiry date.
- **Magic-byte file validation.** Uploaded images are validated at the binary level against the known **magic bytes** of accepted formats (PNG, JPEG, WebP, GIF). Magic bytes are fixed byte sequences at the start of a file that identify its true format independently of the filename or MIME type declared by the client. A file whose binary header does not match its declared type is rejected before reaching storage, preventing MIME-confusion attacks in which a malicious file is disguised as a safe image.

Beyond these three mechanisms, the OWASP coverage includes: broken access control addressed through the RBAC permission model, injection risks eliminated through parameterized database queries throughout the data layer, and security misconfiguration mitigated by a strict CORS policy, security-oriented HTTP headers and mandatory environment-variable validation at application startup.

5.4 VR/AR Immersive Preview

5.4.1 Analysis

Browser-Native Immersive Web. The immersive preview feature is delivered entirely through the browser, with no plugin or companion application required on the user's side. This is made possible by a recent W3C standard that exposes virtual and augmented reality hardware directly to web pages.

WebXR Device API

The **WebXR Device API** [48] is a W3C standard that provides browsers with a unified interface for accessing virtual and augmented reality hardware. It abstracts the differences between headset models, controller layouts and spatial tracking systems, allowing a single web application to deliver immersive experiences across a variety of XR devices without platform-specific adaptations.

Because the API is implemented natively in modern browsers, the immersive preview is accessible to anyone owning a compatible headset, without any installation step.

3D Rendering and Stereo Mode. The asset viewer is powered by **Three.js** [27], a JavaScript library that abstracts the low-level WebGL rendering pipeline. When

a user enters immersive mode, the renderer transitions to **stereoscopic projection**, producing a separate image frame for each eye at the correct inter-pupillary offset to create a convincing perception of depth. In augmented reality mode, the rendered 3D asset is composited over the live camera feed provided by the headset's passthrough cameras, allowing the user to inspect the object within their physical environment.

To maintain rendering correctness in XR sessions, visual post-processing effects that rely on multi-pass framebuffer operations are deactivated during immersive rendering. These effects are not designed for the stereo framebuffer required by headsets and would otherwise cause visual artefacts or rendering errors.

Hardware Validation. The feature was validated on a **Meta Quest 3** [15] mixed-reality headset. The device was connected to the same local network as the development server and accessed the application through its built-in browser over HTTPS, which is required by the WebXR specification. Two modes were tested: a fully immersive **VR mode** in which the user is surrounded by the rendered 3D scene, and a **passthrough AR mode** in which the asset is overlaid on the real environment captured by the device's colour cameras. Both modes operated correctly, confirming that the platform delivers an authentic WebXR experience on consumer mixed-reality hardware.

5.4.2 Realization

On any compatible 3D asset detail page, the user can enter VR or AR mode directly from the browser with a single click. In VR mode, the asset is rendered at true real-world scale inside the headset, allowing buyers to evaluate proportions and surface detail before purchasing. In AR mode, the asset is placed within the user's physical environment, which is particularly useful for architectural models and large-scale environment packs.

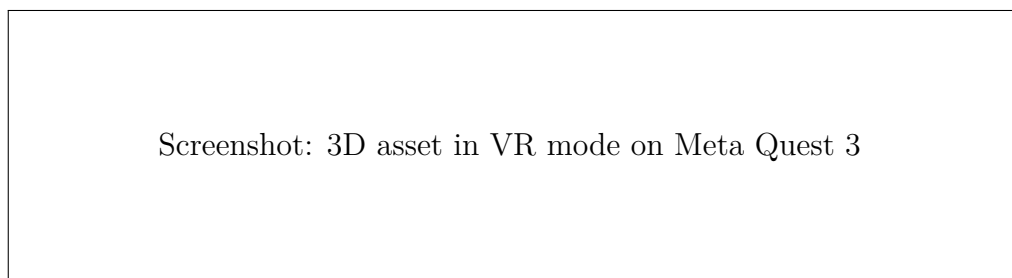


Figure 5.5: WebXR immersive preview — 3D model inspected in VR on Meta Quest 3

5.5 Deployment on Hostinger VPS

The platform is deployed for production on a **Hostinger VPS**. The deployment configuration covers the reverse proxy, the process supervisor and an automated continuous-

delivery workflow that handles every release from source to live server.

5.5.1 Target Infrastructure

Three independent tiers handle different concerns:

- **Hostinger VPS**: hosts the Node.js backend (Express API) and serves the two built React applications (Marketplace and Workspace) as static files.
- **Supabase** (cloud-hosted): handles the PostgreSQL database and the asset file buckets. No database instance is needed on the VPS.
- **Hugging Face Inference API**: serves the LLM and DistilBERT models remotely; no GPU hardware is required on the server.

5.5.2 Key Deployment Concepts

- **Virtual Private Server (VPS)**: an isolated virtual machine running Ubuntu 22.04 LTS, providing full root access without the cost of dedicated hardware.
- **Nginx (reverse proxy)**: terminates HTTPS, serves the static React builds from `/var/www/`, and forwards `/api/*` and `/socket.io` traffic to the Express backend on port 3001.
- **PM2 (process manager)**: keeps the Node.js process alive, restarts it on crash, enables zero-downtime reloads on new deploys, and manages log rotation.
- **Let's Encrypt + Certbot**: free SSL certificates renewed automatically every 90 days; HTTPS is mandatory for WebXR features to work in the browser.
- **Environment variables (.env)**: keep secrets (JWT secret, database credentials, Stripe keys, Hugging Face token) out of the source code and out of the Git history entirely.

5.5.3 CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/cd.yml`) automates the release on every push to `main`:

1. Install dependencies and run lint and integration tests.
2. Build both React frontends with `npm run build`.
3. SSH into the Hostinger VPS, pull the latest code, copy the built `dist/` folders to `/var/www/`, and call `pm2 reload igs-backend` for a zero-downtime restart.

Deployment secrets (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`) are stored in GitHub repository secrets and never appear in workflow logs.

Conclusion

This chapter presented the AI services, the immersive preview feature, and the deployment architecture that complete the platform.

General Conclusion

IGS Marketplace was successfully designed and developed over eight sprints, delivering Tunisia's first dedicated 3D digital asset platform. The system provides a six-level Role-Based Access Control system, an interactive in-browser 3D asset viewer, Stripe payment integration, AI-powered review sentiment analysis, a personalized recommendation engine, a floating chatbot assistant powered by Qwen2.5-72B-Instruct, and a companion internal workspace application — all within a cohesive full-stack ecosystem built on React, Node.js/Express, and PostgreSQL.

Several key technical insights emerged throughout the project. Designing the permission system during Sprint 0 and enforcing it consistently across all subsequent sprints was the single most impactful architectural decision — retrofitting access control would have been significantly more costly. The three-tier layered architecture (Routes → Controllers → Repositories) proved highly maintainable for a REST API at this level of complexity, and applying Scrum as a solo developer, with supervisors occupying the Product Owner and Scrum Master roles, effectively structured six months of work into reviewable, deliverable-focused increments.

Looking ahead, several directions are planned for the platform's continued evolution: a React Native mobile client, a full automated test suite using Jest and Supertest, expanded AI capabilities including streaming chatbot responses and AI-based tag suggestions, a digital asset licensing framework where sellers can define usage rights, and production deployment on a cloud provider with the Docker-based CI/CD pipeline fully active.

Webography

- [1] Unity Technologies. *Unity Asset Store*.
<https://assetstore.unity.com/> [Accessed April 2026]
- [2] Epic Games. *Fab — The unified content marketplace by Epic Games*.
<https://www.fab.com/> [Accessed April 2026]
- [3] Sketchfab (Epic Games). *Sketchfab — Publish & Find 3D Models Online*.
<https://sketchfab.com/> [Accessed April 2026]
- [4] CGTrader. *CGTrader — 3D Model Marketplace and Computer Graphics Forum*.
<https://www.cgtrader.com/> [Accessed April 2026]
- [5] ThreeDy.ai. *Cosmos — AI-powered 3D content platform*.
<https://www.threedy.ai/> [Accessed April 2026]
- [6] Adobe Inc. *Mixamo — Free animated 3D characters*.
<https://www.mixamo.com/> [Accessed April 2026]
- [7] Microsoft. *Visual Studio Code — Code Editing Redefined*.
<https://code.visualstudio.com/> [Accessed May 2026]
- [8] GitHub, Inc. *GitHub — Where the world builds software*.
<https://github.com/> [Accessed May 2026]
- [9] GitHub, Inc. *GitHub Actions Documentation*.
<https://docs.github.com/en/actions> [Accessed May 2026]
- [10] Docker, Inc. *Docker — Accelerated Container Application Development*.
<https://www.docker.com/> [Accessed May 2026]
- [11] Hostinger International. *Hostinger — Web Hosting and VPS*.
<https://www.hostinger.com/> [Accessed May 2026]
- [12] JGraph Ltd. *draw.io — Free Online Diagram Software*.
<https://www.drawio.com/> [Accessed May 2026]
- [13] OpenJS Foundation. *ESLint — Find and fix problems in JavaScript code*.
<https://eslint.org/> [Accessed May 2026]

- [14] Tailwind Labs. *Tailwind CSS — A utility-first CSS framework*.
<https://tailwindcss.com/> [Accessed May 2026]
- [15] Meta Platforms, Inc. *Meta Quest 3 — Mixed Reality Headset*.
<https://www.meta.com/quest/quest-3/> [Accessed May 2026]
- [16] Tool logos used throughout this report were obtained from the official websites of the respective projects: React, Node.js, Express, Vite, PostgreSQL, Supabase, Stripe, Socket.IO, Three.js, Hugging Face, Docker, GitHub, GitHub Actions, draw.io, ESLint, Tailwind CSS, JWT, Hostinger, Meta Quest and L^AT_EX.
<https://react.dev/> ; <https://nodejs.org/> ; <https://threejs.org/> [Accessed May 2026]
- [17] Meta Platforms. *React — The library for web and native user interfaces*.
<https://react.dev/> [Accessed April 2026]
- [18] Vite contributors. *Vite — Next generation frontend tooling*.
<https://vitejs.dev/guide/> [Accessed April 2026]
- [19] OpenJS Foundation. *Node.js Official Documentation*.
<https://nodejs.org/en/docs/> [Accessed April 2026]
- [20] OpenJS Foundation. *Express.js Application Guide*.
<https://expressjs.com/en/guide/routing.html> [Accessed April 2026]
- [21] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.
<https://www.postgresql.org/docs/16/> [Accessed April 2026]
- [22] Supabase Inc. *Supabase Documentation — Storage*.
<https://supabase.com/docs/guides/storage> [Accessed April 2026]
- [23] Supabase Inc. *Supabase Documentation — Auth*.
<https://supabase.com/docs/guides/auth> [Accessed April 2026]
- [24] Stripe, Inc. *Stripe API Reference — Payment Intents*.
https://stripe.com/docs/api/payment_intents [Accessed April 2026]
- [25] Socket.IO contributors. *Socket.IO v4 Documentation*.
<https://socket.io/docs/v4/> [Accessed April 2026]
- [26] Google LLC. *Model Viewer — Web Component for 3D Models*.
<https://modelviewer.dev/> [Accessed April 2026]
- [27] Three.js contributors. *Three.js — JavaScript 3D Library*.
<https://threejs.org/docs/> [Accessed April 2026]

- [28] Hugging Face. *Inference API Documentation*.
<https://huggingface.co/docs/api-inference> [Accessed April 2026]
- [29] Schwaber, K. & Sutherland, J. *The Scrum Guide*. November 2020.
<https://scrumguides.org/scrum-guide.html> [Accessed April 2026]
- [30] Object Management Group. *UML 2.5.1 Specification*. 2017.
<https://www.omg.org/spec/UML/2.5.1> [Accessed April 2026]
- [31] Jones, M. et al. *RFC 7519 — JSON Web Token (JWT)*. IETF, 2015.
<https://datatracker.ietf.org/doc/html/rfc7519> [Accessed April 2026]
- [32] Ferraiolo, D. & Kuhn, R. *Role-Based Access Control*. NIST, 2009.
- [33] OWASP Foundation. *OWASP Top Ten — Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/> [Accessed April 2026]
- [34] P Market Research. *Worldwide 3D Marketplace Models Market Research 2024 — by Type, Application, Participants and Countries, Forecast to 2030*. 2024.
<https://pmarketresearch.com/worldwide-3d-marketplace-models-market-research-2024>
[Accessed April 2026]
- [35] Citius Research. *3D Project Market Report*. 2024.
<https://citiusresearch.com/publication-details/3d-project-market-report> [Accessed April 2026]
- [36] Roques, P. *UML 2 par la Pratique — Études de cas et exercices corrigés*. 8^e édition. Eyrolles, 2014.
- [37] Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- [38] Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- [39] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<https://scikit-learn.org/> [Accessed May 2026]
- [40] Vaswani, A. et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
<https://arxiv.org/abs/1706.03762> [Accessed May 2026]
- [41] Qwen Team, Alibaba Cloud. *Qwen2.5: A Party of Foundation Models*. 2024.
<https://huggingface.co/Qwen/Qwen2.5-72B-Instruct> [Accessed May 2026]

- [42] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33.
<https://arxiv.org/abs/2005.11401> [Accessed May 2026]
- [43] Devlin, J. et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
<https://arxiv.org/abs/1810.04805> [Accessed May 2026]
- [44] Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.
<https://arxiv.org/abs/1910.01108> [Accessed May 2026]
- [45] Socher, R. et al. (2013). Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. *Proceedings of EMNLP 2013*.
<https://nlp.stanford.edu/sentiment/> [Accessed May 2026]
- [46] Lops, P., de Gemmis, M. & Semeraro, G. (2011). Content-based Recommender Systems: State of the Art and Trends. In F. Ricci et al. (Eds.), *Recommender Systems Handbook*. Springer.
- [47] Jaccard, P. (1912). The distribution of the flora in the alpine zone. *New Phytologist*, 11(2), 37–50.
- [48] W3C. *WebXR Device API — W3C Candidate Recommendation*. 2024.
<https://www.w3.org/TR/webxr/> [Accessed May 2026]

Abstract

This end-of-studies project presents the design and development of **IGS Marketplace**, a full-stack digital platform built for **Inherited Games Studio (IGS)**, a Tunisian video-game company. The solution addresses the absence of a national marketplace for 3D creative content by offering local creators a dedicated space to publish and monetize their work, with secure payments in both international and Tunisian currency.

The platform consists of two complementary applications sharing a common back-end: a public *Marketplace* covering thirteen asset categories with interactive 3D and WebXR previews on virtual-reality headsets, and an internal *Workspace* for project management, content moderation and administration. Three AI services are integrated — a conversational chatbot, a DistilBERT-based review sentiment analyzer and a Jaccard-based recommendation engine. The technical stack relies on **React, Node.js / Express** and **PostgreSQL (Supabase)**, following the **Scrum** methodology over five sprints.

Keywords: 3D Marketplace, WebXR, React, Node.js, PostgreSQL, RBAC, Scrum, Artificial Intelligence.

Résumé

Ce projet de fin d'études présente la conception et le développement de **IGS Marketplace**, une plateforme web full-stack réalisée pour **Inherited Games Studio (IGS)**, société tunisienne de développement de jeux vidéo. La solution répond à l'absence d'une marketplace nationale dédiée aux contenus 3D, en offrant aux créateurs locaux un espace de publication et de monétisation avec un paiement sécurisé en devises internationales et en dinar.

La plateforme se compose de deux applications partageant un back-end commun: une *Marketplace* publique couvrant treize catégories d'assets avec prévisualisation 3D interactive et mode immersif WebXR sur casques de réalité virtuelle, et un *Workspace* interne pour la gestion de projet, la modération et l'administration. Trois services d'intelligence artificielle sont intégrés — un chatbot conversationnel, une analyse de sentiment basée sur DistilBERT et un moteur de recommandations reposant sur la similarité de Jaccard. La pile technique repose sur **React, Node.js / Express** et **PostgreSQL (Supabase)**, suivant la méthodologie **Scrum** sur cinq sprints.

Mots-clés : Marketplace 3D, WebXR, React, Node.js, PostgreSQL, RBAC, Scrum, Intelligence Artificielle.

ملخص

يقدم هذا المشروع تصميم وتطوير منصة **IGS Marketplace** لفائدة شركة **Studio Games Inherited** التونسية المتخصصة في تطوير ألعاب الفيديو. تعالج المنصة غياب فضاء وطني مخصص للأصول الإبداعية ثلاثية الأبعاد، وتوفر للمبدعين المحليين فضاء للنشر والبيع مع دعم الدفع الآمن بالعملات الدولية وبالدينار التونسي.

تتكون المنصة من تطبيقين متكاملين يشتركان في خادم خلفي موحد: واجهة عامة **Marketplace** تغطي ثلاث عشرة فئة من الأصول الرقمية مع عرض تفاعلي ومعاينة غامرة عبر تقنية **WebXR**، ولوحة تحكم داخلية **Workspace** لإدارة المشاريع ومراقبة المحتوى. تدمج المنصة ثلاث خدمات للذكاء الاصطناعي: روبوت محادثة، ومحلل لمشاعر التقييمات يعتمد على نموذج **DistilBERT** ومحرك توصيات مبني على مقياس **Jaccard**. وتعتمد التقنيات المستخدمة على **React** و **Node.js / Express** و **PostgreSQL (Supabase)**، باتباع منهجية **Scrum** عبر خمس دورات تطويرية.

الكلمات المفتاحية: منصة ثلاثية الأبعاد، **WebXR**، **React**، **Node.js**، **PostgreSQL**، **RBAC**، **Scrum**، ذكاء اصطناعي.
